RAG & AI Agents Made Simple

Retrieval, LangChain, LangGraph, LangSmith,
agent evaluation and human-in-the-loop

Naive · Advanced · Hybrid · Graph · Agentic · Corrective · Self · Adaptive RAG
LangChain · LangGraph · LangSmith · AI agents · Multi-agent systems
Evaluation & testing · Human-in-the-loop · Guardrails

Prepared by Rahul Kothuri

How to use this guide

This is the third guide in the series after *Machine Learning Made Simple* and *Deep Learning Made Simple*. It covers how to build applications *on top of* large language models: grounding them in your own data (RAG), turning them into agents that use tools, wiring everything with LangChain and LangGraph, and proving it works with LangSmith, evaluations, tests and human oversight.

Each concept follows the same pattern: **plain-words idea → diagram → only the formulas or code that matter**. Code snippets are short *sketches* that show the shape of the API, not complete programs.

Box	What it holds
The idea in plain words	the concept without jargon
Real-life analogy	something familiar to anchor it
Key equations	formulas worth knowing, every symbol explained
Worked example / When to use it	numbers and practical guidance
Watch out	common mistakes and limits

A note on versions. LangChain, LangGraph and LangSmith change quickly. This guide follows the LangChain 1.x style (`create_agent`, middleware) and current LangGraph concepts (`StateGraph`, checkpointers, `interrupt` / `Command`). Always check the official documentation for exact function names before you code.

Contents

How to use this guide **1**

1 Why RAG? LLM Limits & the Big Picture **4**
1.1 What LLMs can't do on their own . . . 4
1.2 The RAG idea . . . 4
1.3 RAG vs fine-tuning vs prompting . . . 5
1.4 Where this guide goes . . . 5

2 The RAG Pipeline: Indexing **6**
2.1 Step 1 – Load and clean . . . 6
2.2 Step 2 – Chunking . . . 6
2.3 Step 3 – Embeddings . . . 7
2.4 Step 4 – Vector databases . . . 9

3 Retrieval & Generation **11**
3.1 Dense (vector) search . . . 11
3.2 Sparse (keyword) search: BM25 . . . 11
3.3 Hybrid search . . . 11
3.4 Metadata filtering . . . 12
3.5 Re-ranking . . . 12
3.6 MMR – diverse results . . . 13
3.7 Generation: building the prompt . . . 13

4 Types of RAG **15**
4.1 Naive RAG . . . 15
4.2 Advanced RAG . . . 15
4.3 Modular RAG . . . 16
4.4 Hybrid RAG . . . 16
4.5 Graph RAG . . . 17
4.6 Agentic RAG . . . 18
4.7 Corrective RAG (CRAG) . . . 19
4.8 Self-RAG . . . 19
4.9 Adaptive RAG . . . 20
4.10 Conversational RAG . . . 20
4.11 Multimodal RAG . . . 20
4.12 Other patterns worth knowing . . . 20
4.13 Comparison of RAG types . . . 21

5 LangChain **22**
5.1 The LangChain ecosystem . . . 22
5.2 Core components . . . 22
5.3 LCEL and Runnables . . . 23
5.4 Agents in LangChain . . . 23
5.5 Memory . . . 24

6 AI Agents **25**
6.1 What is an AI agent? . . . 25
6.2 Anatomy of an agent . . . 25
6.3 Tool calling . . . 25

6.4 The ReAct pattern . . . 27
6.5 Common agent and workflow patterns . . . 27
6.6 Multi-agent systems . . . 28
6.7 Reliability: retries, errors and limits . . . 28

7 LangGraph 29
7.1 What LangGraph is . . . 29
7.2 Core concepts . . . 29
7.3 The agent loop as a graph . . . 30
7.4 A Corrective-RAG graph . . . 30
7.5 Persistence and checkpoints . . . 31
7.6 Human-in-the-loop in LangGraph . . . 31

8 LangSmith: Observability & Evaluation Platform 33
8.1 Why observability? . . . 33
8.2 Tracing . . . 33
8.3 What LangSmith offers . . . 34
8.4 The evaluation workflow . . . 34

9 Evaluating & Testing RAG and Agents 35
9.1 Why evaluation is different for LLM apps . . . 35
9.2 Evaluating RAG . . . 35
9.3 LLM-as-judge . . . 36
9.4 Evaluating agents . . . 37
9.5 Testing strategy . . . 38

10 Human-in-the-Loop, Guardrails & Production 40
10.1 Human-in-the-loop (HITL) . . . 40
10.2 Guardrails and security . . . 41
10.3 Production checklist . . . 42

11 Cheat Sheet & Interview Revision 43
11.1 Formula sheet . . . 43
11.2 Which RAG or agent design? . . . 43
11.3 Interview quick revision . . . 43

Further Reading 45

CHAPTER 1

Why RAG? LLM Limits & the Big Picture

1.1 What LLMs can't do on their own

An LLM stores what it learned during training inside its weights ("parametric knowledge"). That causes three problems in a company setting:

1. **Hallucination** – when it doesn't know, it may still produce a confident, plausible but wrong answer.
2. **Stale knowledge** – it knows nothing after its training cut-off (new policies, this month's prices).
3. **No private data** – it has never seen your HR handbook, contracts, tickets or database.

1.2 The RAG idea

The idea in plain words

Retrieval-Augmented Generation: before the LLM answers, **search** your own documents for the most relevant passages and **paste them into the prompt**. The model then answers from that evidence – like an open-book exam – and can cite its sources.

Real-life analogy

A closed-book exam tests memory; an open-book exam lets you look up the right page first. RAG turns every question into an open-book exam, and the retriever is the student's skill at finding the right page quickly.

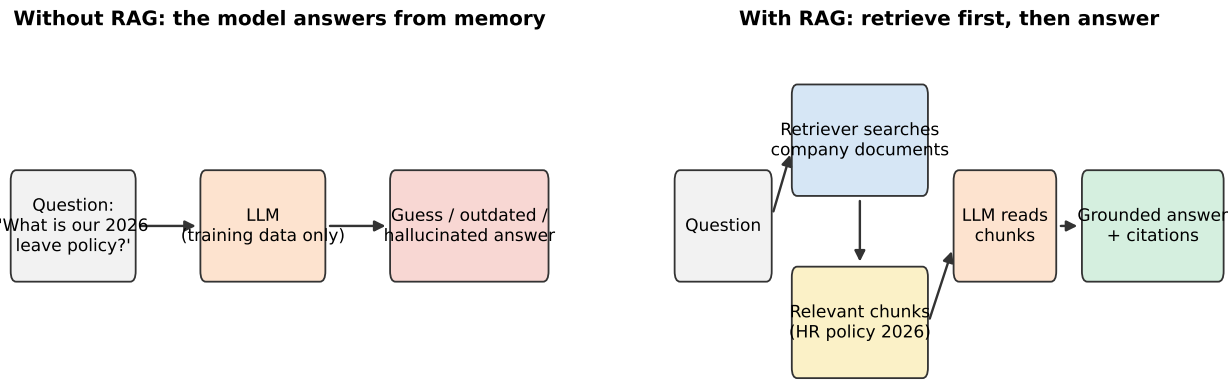


Figure 1.1: Without RAG the model guesses from memory; with RAG it answers from retrieved evidence.

Key equations

RAG in one line: $\text{answer} = \text{LLM}(\text{instructions} + \text{retrieve}(q,\ k) + q)$

q = the user question, k = number of chunks retrieved ("top-k", typically 3–10).

1.3 RAG vs fine-tuning vs prompting

	Prompt engineering	RAG	Fine-tuning
Changes	The instructions	The *knowledge* given at run time	The model's *weights / behaviour*
Best for	Format, tone, simple tasks	Facts, private and fast-changing data, citations	Style, domain jargon, a narrow task format
Update knowledge	Edit prompt	Re-index documents (minutes)	Retrain (hours/days, cost)
Hallucination control	Low	High (grounded + citable)	Medium
Cost	Lowest	Moderate (vector DB + longer prompts)	Highest up front

Remember

Rule of thumb: use **RAG for knowledge** and **fine-tuning for behaviour**. They can be combined. Always try good prompting + RAG first.

1.4 Where this guide goes

1. **Chapters 2–3** – how a RAG system works: chunking, embeddings, vector databases, keyword/hybrid search, re-ranking.
2. **Chapter 4** – all the RAG *types*: naive, advanced, modular, hybrid, Graph RAG, agentic, corrective, self, adaptive, conversational and multimodal.
3. **Chapters 5–7** – LangChain, AI agents and LangGraph.
4. **Chapters 8–9** – LangSmith, then evaluation and testing of RAG systems and agents.
5. **Chapter 10** – human-in-the-loop, guardrails, production checklist and interview revision.

CHAPTER 2

The RAG Pipeline: Indexing

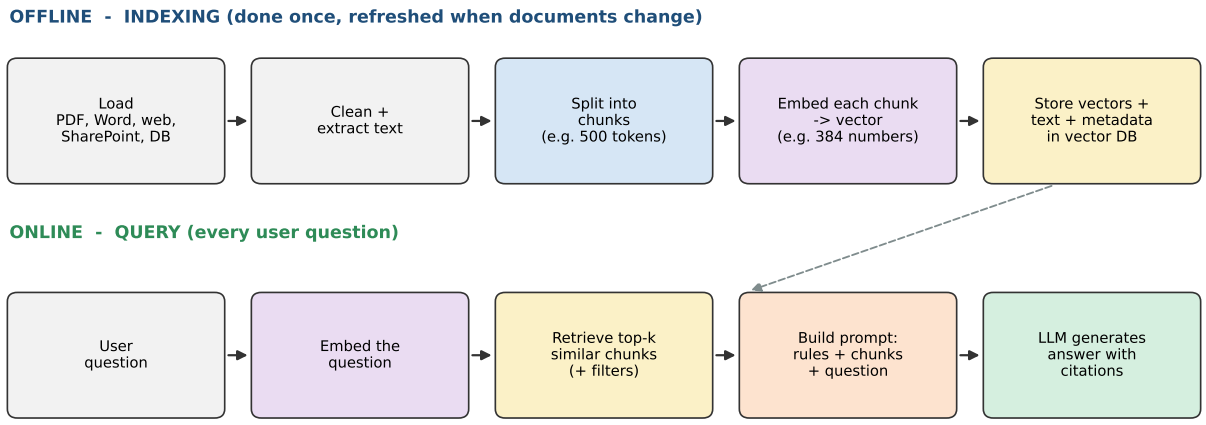


Figure 2.1: Every RAG system has two halves: an offline indexing pipeline and an online query pipeline.

2.1 Step 1 – Load and clean

Document loaders read PDFs, Word, HTML, Markdown, CSV, SharePoint, Confluence, databases. Clean the text: remove headers/footers, page numbers, navigation menus, duplicated boilerplate. Keep **metadata** – source file, page, section title, date, department, access level. Metadata is later used for filtering, citations and permissions.

Watch out

Tables and scanned PDFs are the most common source of bad RAG answers. Use a layout-aware parser or OCR, and check what the extracted text actually looks like before blaming the model.

2.2 Step 2 – Chunking

The idea in plain words

Documents are too long to embed or paste whole, so we cut them into **chunks**. A good chunk is small enough to be about one thing (so search is precise) but big enough to make sense on its own (so the LLM gets full context).

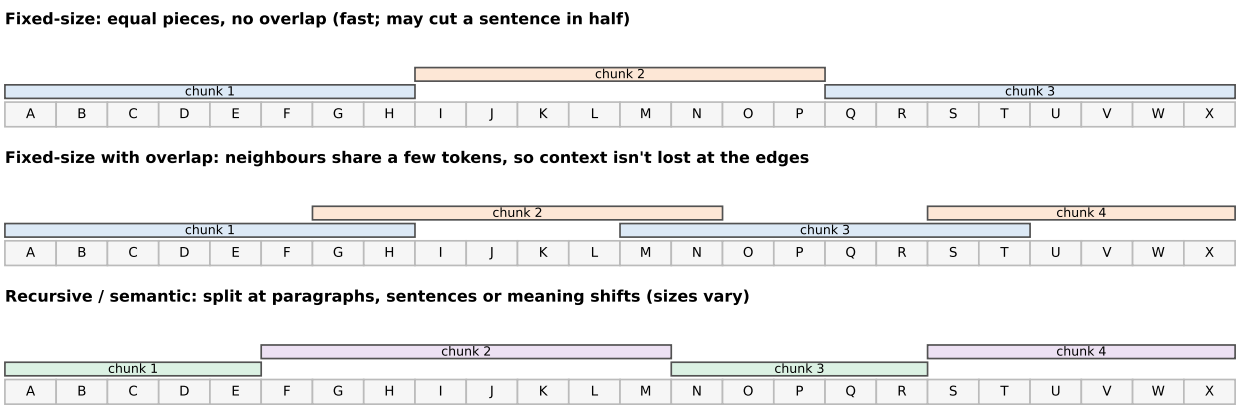


Figure 2.2: Three common chunking strategies.

Strategy	How it works	Good for
Fixed-size	Every N characters/tokens	Quick baselines, uniform text
Fixed + overlap	Neighbouring chunks share e.g. 10–20%	Avoids losing sentences at boundaries
Recursive	Split on paragraphs, then lines, then sentences, until under the size limit (LangChain's `RecursiveCharacterTextSplitter`)	The sensible default
Structure-aware	Split by headings, Markdown/HTML sections, code functions, table rows	Manuals, docs, code
Semantic	Split where the embedding meaning changes sharply	Long narrative text
Parent–child (small-to-big)	Search small chunks, return the larger parent	Precise search + rich context

Key equations

Number of chunks with overlap: $n \approx \left\lceil \frac{L-o}{s-o} \right\rceil$ L = document length, s = chunk size, o = overlap (same units).

Worked example

A 10,000-token policy, chunk size 500, overlap 100: $n \approx \lceil (10000-100)/(500-100) \rceil = \lceil 24.75 \rceil = 25$ chunks (vs 20 with no overlap – overlap costs a little extra storage).

2.3 Step 3 – Embeddings

The idea in plain words

An **embedding model** turns a piece of text into a list of numbers (a vector, e.g. 384, 768 or 1536 values). Texts with similar *meaning* get vectors that point in similar directions – even if they share no words ("annual leave" vs "vacation days").

Key equations

Cosine similarity (the usual score): $\cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{\|\mathbf{q}\|\,\|\mathbf{d}\|} = \frac{\sum_i q_i d_i}{\sqrt{\sum_i q_i^2}\sqrt{\sum_i d_i^2}}$

Range -1 to 1; higher = more similar. For normalised vectors (length 1) cosine equals the plain dot product. Some stores use distance instead (lower = closer).

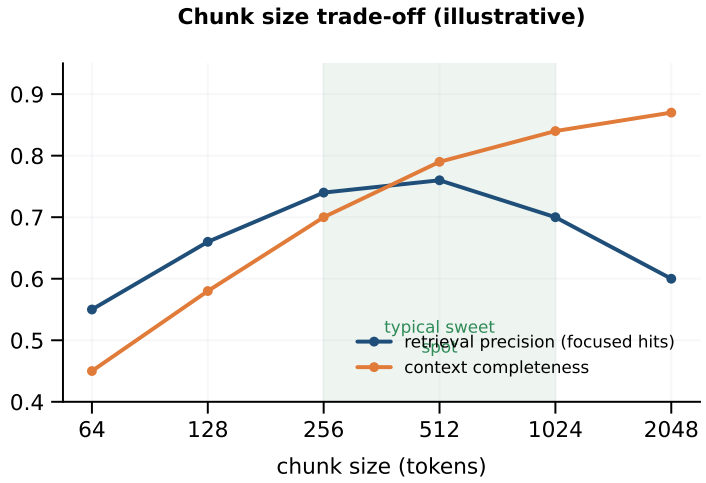


Figure 2.3: Smaller chunks give sharper hits but lose context; larger chunks keep context but dilute the match. Tune with evaluation, not guesswork (illustrative).

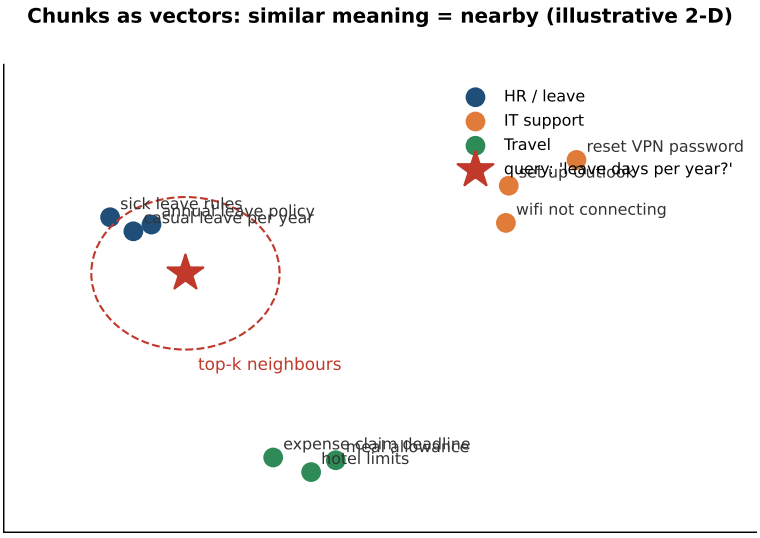


Figure 2.4: Chunks about the same topic cluster together; the question lands near the chunks that can answer it.

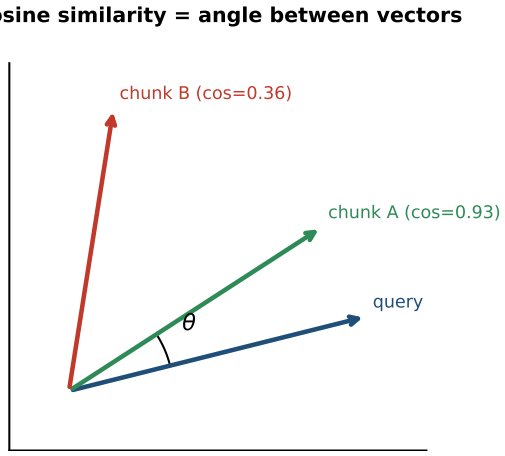


Figure 2.5: Cosine similarity only cares about direction, not length.

Worked example

$\mathbf{q} = (1, 2, 0)$, $\mathbf{d} = (2, 3, 1)$: $\mathbf{q} \cdot \mathbf{d} = 2 + 6 + 0 = 8$; $\|\mathbf{q}\| = \sqrt{5} = 2.24$; $\|\mathbf{d}\| = \sqrt{14} = 3.74$; cosine $= 8/(2.24 \times 3.74) = 0.96$ – very similar.

- **Popular models**: OpenAI `text-embedding-3-small/large`, Google Gemini embeddings, Cohere, open-source `all-MiniLM-L6-v2` (384 dims, fast), BGE, E5 and multilingual models.
- Use the **same** embedding model for documents and queries. Changing the model means re-embedding everything.
- Bigger isn't always better: compare on your own data (retrieval recall), plus cost and latency.

2.4 Step 4 – Vector databases

A vector store keeps (vector, text, metadata) and answers "give me the k nearest vectors to this one", usually with metadata filters.

Store	Notes
FAISS	Library (not a server); very fast, in-memory, great for prototypes
Chroma	Simple local/embedded DB; popular for learning and small apps
pgvector	Vectors inside PostgreSQL – one database for everything
Pinecone, Weaviate, Qdrant, Milvus	Managed/scalable vector databases; hybrid search, filtering
Azure AI Search, Elastic-search/OpenSearch	Enterprise search engines with keyword + vector + hybrid in one

2.4.1 How search stays fast: approximate nearest neighbours

Comparing the query with millions of vectors one by one is slow, so stores build an **ANN index** that trades a tiny bit of accuracy for huge speed.

- **Flat** – exact, brute force; fine up to tens of thousands of vectors.
- **HNSW** – layered graph; the most common default; fast and accurate, uses more memory.
- **IVF** – cluster vectors, then search only the closest clusters; good for very large collections. Often combined with **product quantisation (PQ)** to compress vectors.

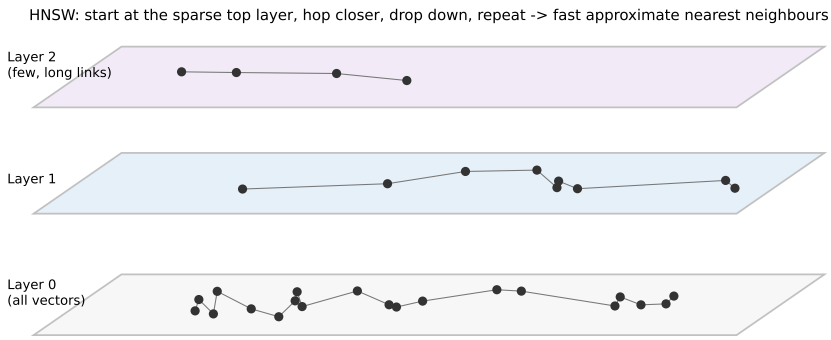


Figure 2.6: HNSW (Hierarchical Navigable Small World) graph: navigate from coarse to fine layers.

CHAPTER 3

Retrieval & Generation

3.1 Dense (vector) search

Embed the question, find the top-k most similar chunks. **Strength**: understands meaning and synonyms. **Weakness**: can miss exact terms – product codes ("XPS-9530"), error numbers, names, acronyms.

3.2 Sparse (keyword) search: BM25

The idea in plain words

The classic search-engine score. A chunk ranks high if it contains the query's words, especially *rare* words, several times – but extra repeats help less and less, and long chunks are penalised a little.

Key equations

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

$f(t,d)$ = times term t appears in chunk d; $|d|$ = chunk length; avgdl = average length; $k_1 \approx 1.2$ (saturation), $b \approx 0.75$ (length normalisation); IDF is high for rare terms.

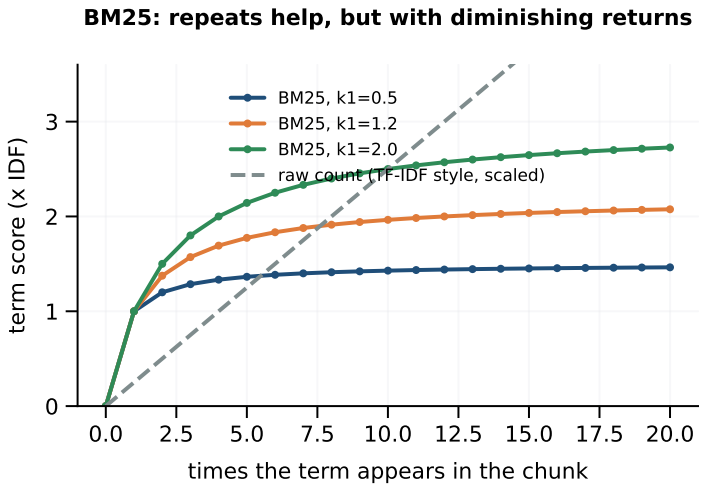


Figure 3.1: With BM25, the 10th repeat of a word adds far less than the first.

3.3 Hybrid search

The idea in plain words

Run **both** keyword and vector search and merge the results. Keywords catch exact codes and names; vectors catch meaning. Hybrid search is one of the most reliable upgrades for real-world RAG.

Key equations

Reciprocal Rank Fusion (RRF): $\mathrm{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \mathrm{rank}_r(d)}, \quad k \approx 60$

Uses only rank positions, so different score scales don't matter.

Weighted score fusion: $s(d) = \alpha \cdot s_{\text{vector}}(d) + (1 - \alpha) \cdot s_{\text{BM25}}(d)$ (scores normalised first, $\alpha \in [0, 1]$).

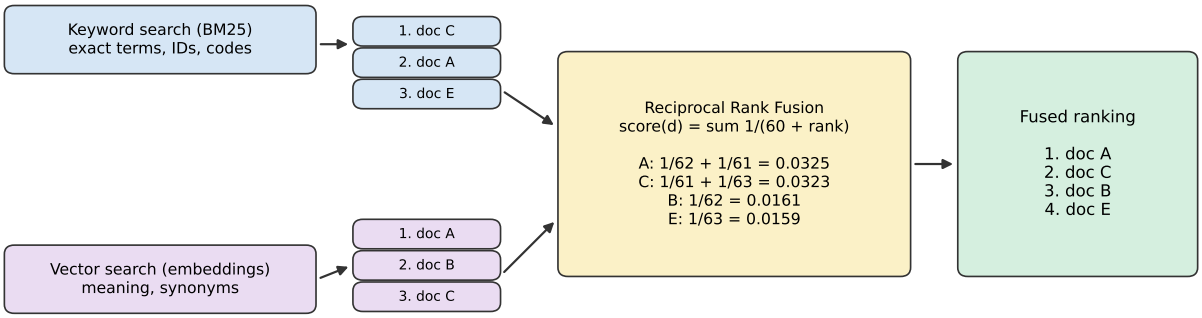


Figure 3.2: Worked RRF example: doc A is 2nd in keyword and 1st in vector search, so it wins overall.

3.4 Metadata filtering

Restrict search *before* ranking: `department = "Finance"`, `year >= 2026`, `country = "IN"`, `access_group in user.groups`. This improves precision and enforces **permissions** (a user must never retrieve documents they aren't allowed to see).

3.5 Re-ranking

The idea in plain words

First stage: fetch many candidates fast (e.g. top 50). Second stage: a slower, smarter **re-ranker** reads each (question, chunk) pair together and re-orders them; keep only the best 3–5 for the prompt.

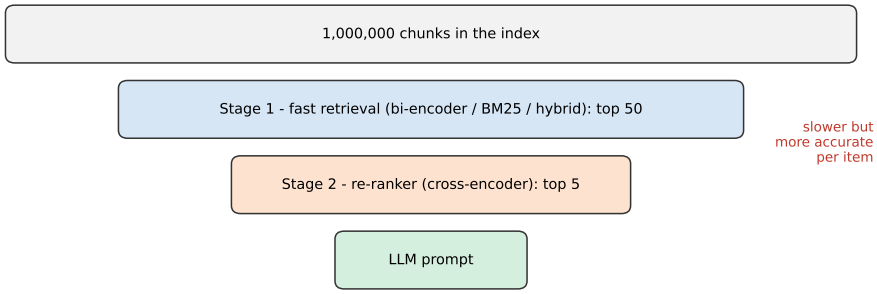


Figure 3.3: Retrieve wide and cheap, then re-rank narrow and precise.

Examples: Cohere Rerank, BGE-reranker, `ms-marco` cross-encoders, or an LLM asked to score relevance.

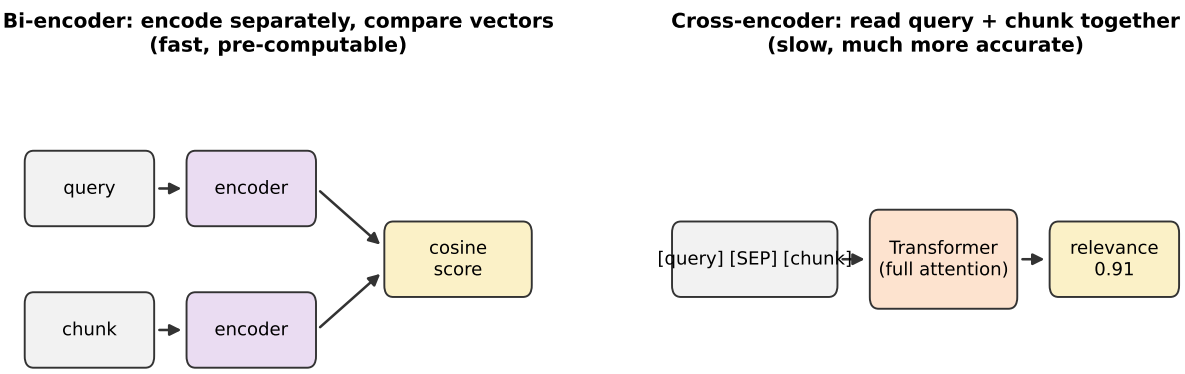


Figure 3.4: Bi-encoders embed separately (fast, used for the index); cross-encoders read both texts together (accurate, used for re-ranking).

3.6 MMR – diverse results

The idea in plain words

Top-k often returns near-duplicates (three copies of the same paragraph). **Maximal Marginal Relevance** picks chunks that are relevant to the query but *different* from those already picked.

Key equations

$\text{MMR} = \arg\max_{d \in R \setminus S} \left[\lambda \cdot \text{sim}(d, q) - (1 - \lambda) \cdot \max_{s \in S} \text{sim}(d, s) \right]$

S = already selected, $\lambda = 1$ for pure relevance, lower for more diversity (e.g. 0.5).

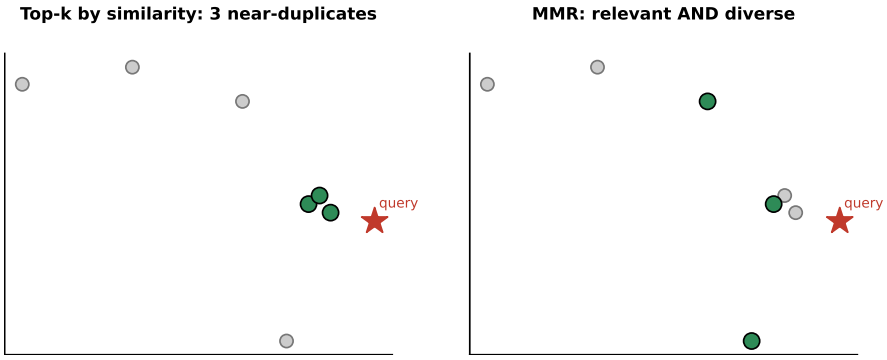


Figure 3.5: MMR replaces duplicates with diverse, still-relevant chunks.

3.7 Generation: building the prompt

Python (sketch)

```
SYSTEM: You are an HR assistant. Answer ONLY from the context.
If the answer is not in the context, say "I don't know".
Cite sources as [doc, page].

CONTEXT:
[HR_Policy_2026.pdf, p4] Employees get 18 days of annual leave...
[HR_Policy_2026.pdf, p5] Up to 10 unused days carry over...

QUESTION: How many leave days can I carry forward?
```

- **Grounding instructions**: answer only from context, admit "I don't know", cite sources.

- **Order matters**: LLMs attend most to the start and end of long contexts ("lost in the middle"). Put the best chunks first, and send fewer, better chunks.
- **Context budget**: $k\times$ chunk size + question + instructions must fit the context window, and longer prompts cost more and are slower.
- **Citations**: keep chunk IDs and metadata so the UI can link to the source.

Watch out

Retrieved text is *data*, not instructions. A document containing "ignore previous instructions" is an **indirect prompt injection**. Keep the system prompt authoritative, and never give a RAG app more tool permissions than it needs.

CHAPTER 4

Types of RAG

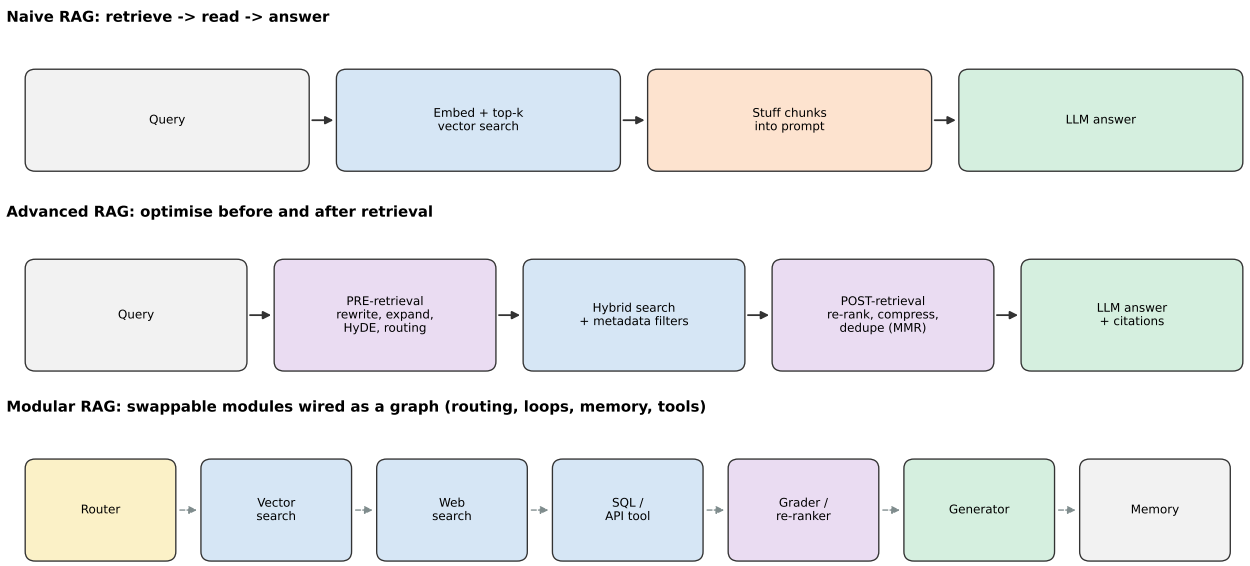


Figure 4.1: The three generations of RAG architecture.

4.1 Naive RAG

The idea in plain words

The basic recipe: chunk → embed → top-k vector search → stuff into prompt → answer.

Quick to build and a good baseline. Typical failures: misses exact terms, retrieves duplicates or irrelevant chunks, struggles with vague, multi-part or follow-up questions, and has no way to notice that retrieval failed.

4.2 Advanced RAG

The idea in plain words

Keep the same pipeline but add improvements **before** retrieval (make the query better) and **after** retrieval (make the context better).

Stage	Techniques
Indexing	Better chunking, metadata, summaries per chunk, parent–child, hypothetical questions per chunk
Pre-retrieval	Query rewriting, multi-query, HyDE, decomposition, step-back questions, routing to the right index
Retrieval	Hybrid search, metadata filters, fine-tuned embeddings
Post-retrieval	Re-ranking, MMR de-duplication, contextual compression (keep only relevant sentences), reordering
Generation	Grounding prompts, citations, answer verification

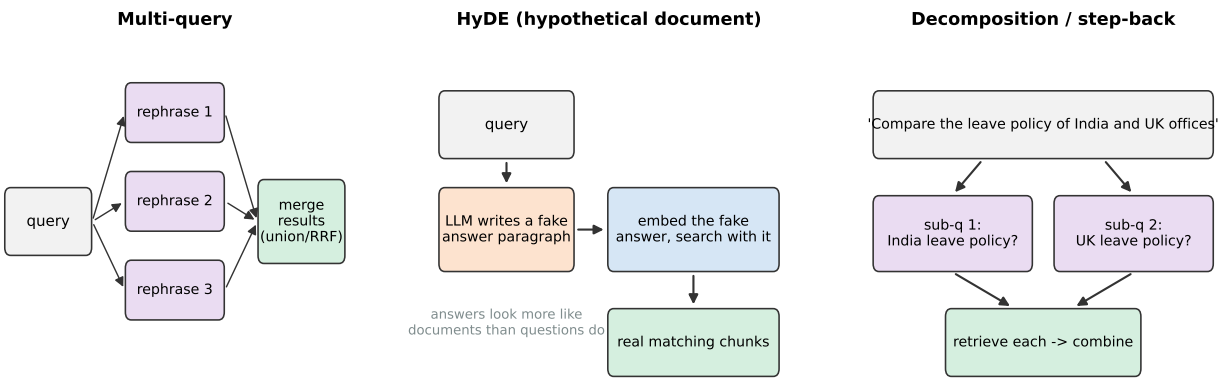


Figure 4.2: Three ways to improve the query before searching.

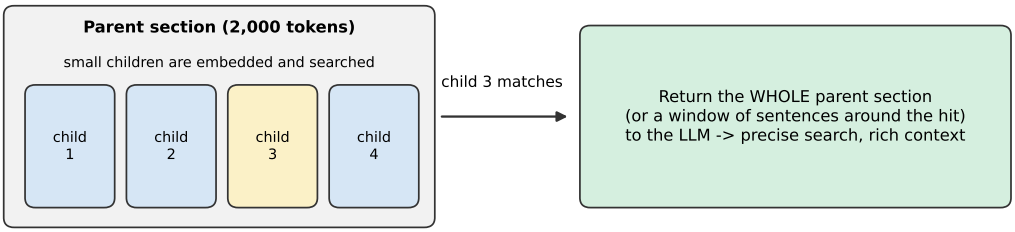


Figure 4.3: Small-to-big (parent document) retrieval: search small, return big. The "sentence window" variant returns a few sentences around each hit.

4.3 Modular RAG

Breaks the pipeline into interchangeable modules – routers, retrievers, re-rankers, graders, memory, tools, generators – connected as a graph with branches and loops. Agentic, corrective and adaptive RAG are all modular RAG; LangGraph is a natural way to build them.

4.4 Hybrid RAG

The idea in plain words

Combine more than one retrieval method – most often **keyword (BM25) + vector**, and sometimes a **knowledge graph** too – then fuse and re-rank the results.

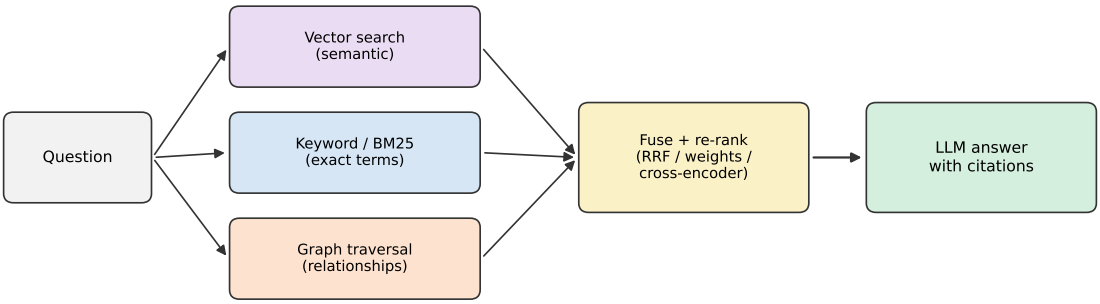


Figure 4.4: Hybrid RAG fuses several retrievers.

When to use it

Enterprise content full of IDs, codes, product names and acronyms; legal/technical documents; any time pure vector search misses exact matches.

4.5 Graph RAG

The idea in plain words

Plain RAG retrieves isolated text chunks. **Graph RAG** first builds a **knowledge graph** – *entities* (people, products, teams, suppliers) as nodes and *relationships* as edges – then retrieves by walking that graph. It can answer questions that need **connections across many documents**.

Real-life analogy

Vector search is like finding pages that mention a name. A knowledge graph is the organisation chart and project map: it lets you follow "who manages whom, which project uses which tool" across the whole company.

A knowledge graph: entities (nodes) + relationships (edges) [illustrative]

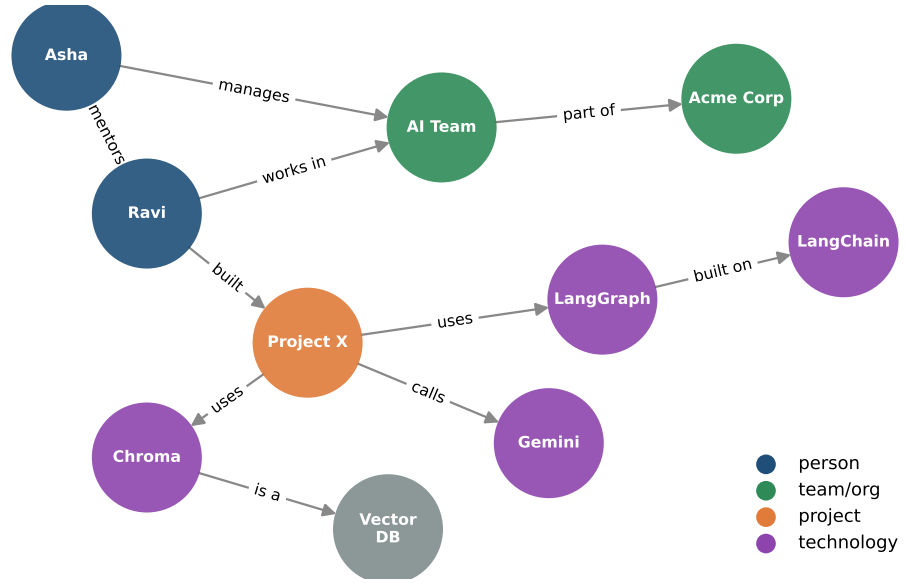


Figure 4.5: A small knowledge graph. "Which technologies does Ravi's project depend on?" needs two hops: Ravi → Project X → LangGraph, Chroma, Gemini.

- **Triples**: knowledge is stored as (subject, relation, object), e.g. (Project X, uses, LangGraph).
- **Graph stores and queries**: Neo4j (Cypher), Amazon Neptune, or in-memory NetworkX; an LLM can translate questions into graph queries (text-to-Cypher).
- **Local search** answers questions about specific entities by expanding their neighbourhood (multi-hop).
- **Global search** answers "big picture" questions ("what are the main risks across all audit reports?") by summarising communities of related entities – something top-k chunks can't do.

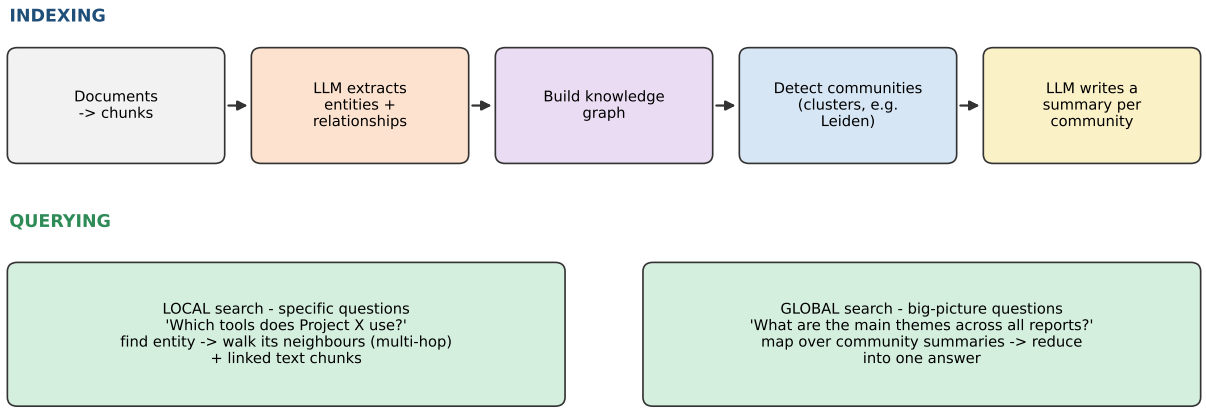


Figure 4.6: GraphRAG pipeline (in the style of Microsoft's GraphRAG): extraction, graph building, community summaries, then local or global search.

	Vector RAG	Graph RAG
Unit retrieved	Text chunks	Entities, relationships, community summaries (+ chunks)
Good at	Direct factual look-ups	Multi-hop, relationship and whole-corpus questions
Indexing cost	Low (embed once)	High (many LLM calls to extract entities)
Maintenance	Easy re-index	Graph must be updated as data changes
Explainability	Shows source chunks	Shows the path of relationships

4.6 Agentic RAG

The idea in plain words

Put an **agent** in charge of retrieval. Instead of always doing one search, the LLM decides *whether* to retrieve, *which* source or tool to use, and whether to search again after reading the results.

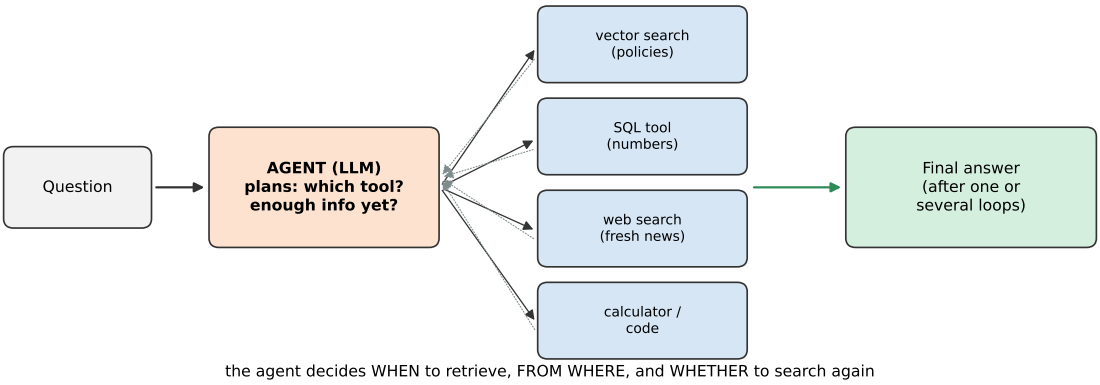


Figure 4.7: Agentic RAG: retrieval is one of several tools the agent can choose, repeatedly.

When to use it

Questions that need several sources (documents + SQL + web), multi-step research, or follow-up searches. Cost: more LLM calls, more latency, and it needs step limits and good evaluation.

4.7 Corrective RAG (CRAG)

The idea in plain words

After retrieving, **grade** the documents. If they are good, refine and use them; if unclear, add a web search; if bad, throw them away and search elsewhere. Never answer from evidence you know is poor.

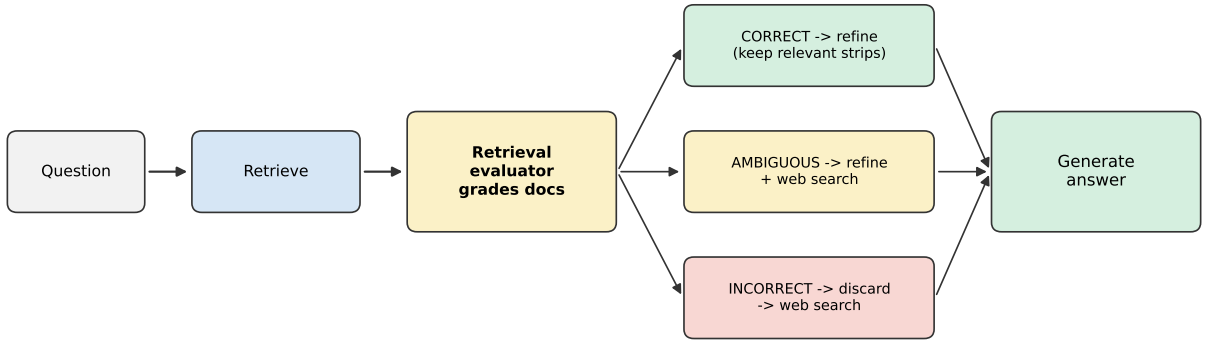


Figure 4.8: Corrective RAG adds a quality check between retrieval and generation.

Works with any frozen LLM (no training needed) – easy to build in LangGraph as *retrieve* → *grade* → *(generate / rewrite query* → *web search* → *generate)*.

4.8 Self-RAG

The idea in plain words

The model itself is **trained** to decide when to retrieve and to critique its own work using special *reflection tokens*: is retrieval needed? is this passage relevant? is my sentence supported? is the answer useful?

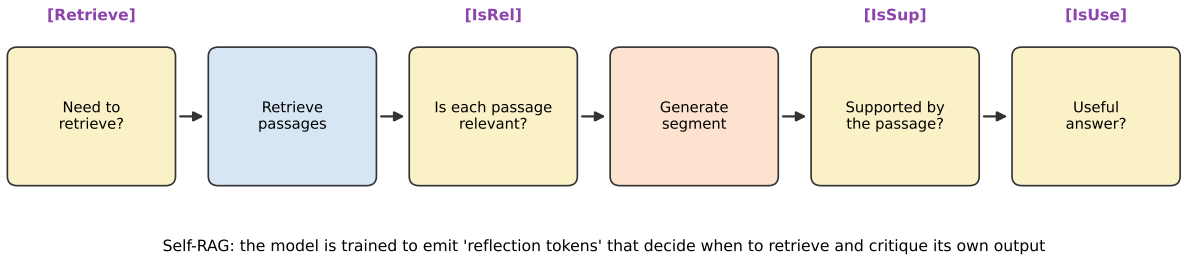


Figure 4.9: Self-RAG embeds retrieval decisions and self-critique into the model.

Remember

Self-RAG vs CRAG: Self-RAG builds the checks *into the model* (needs special training). CRAG wraps checks *around any model* as a pipeline – usually more practical in production. Many teams build a "self-reflective" RAG in LangGraph with an LLM grader instead of training.

4.9 Adaptive RAG

The idea in plain words

Not every question needs the same effort. A **router** classifies the question and picks a strategy: answer directly, do one retrieval, or run multi-step agentic retrieval.

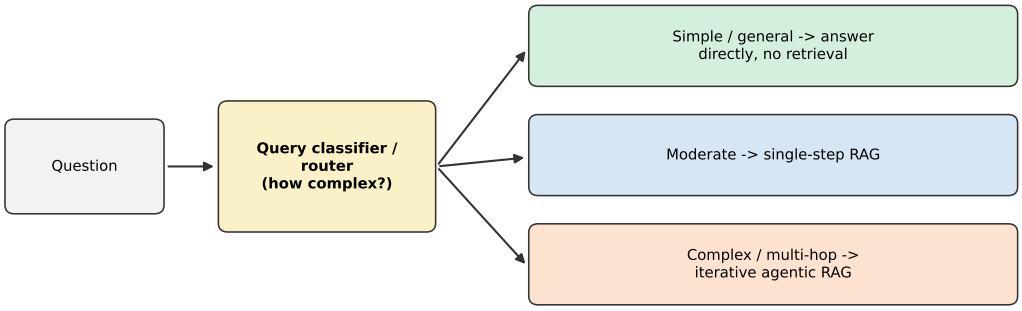


Figure 4.10: Adaptive RAG saves cost on easy questions and spends effort on hard ones.

4.10 Conversational RAG

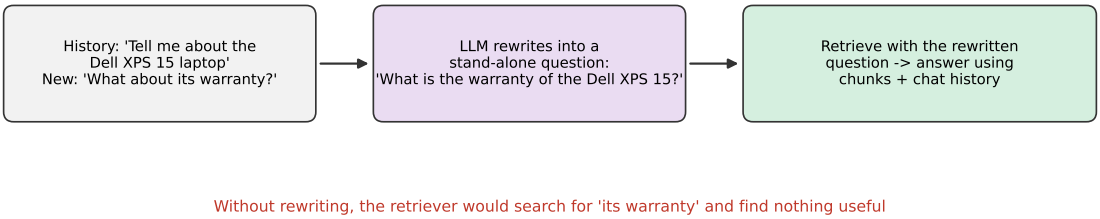


Figure 4.11: Follow-up questions must be rewritten into stand-alone questions before searching.

4.11 Multimodal RAG

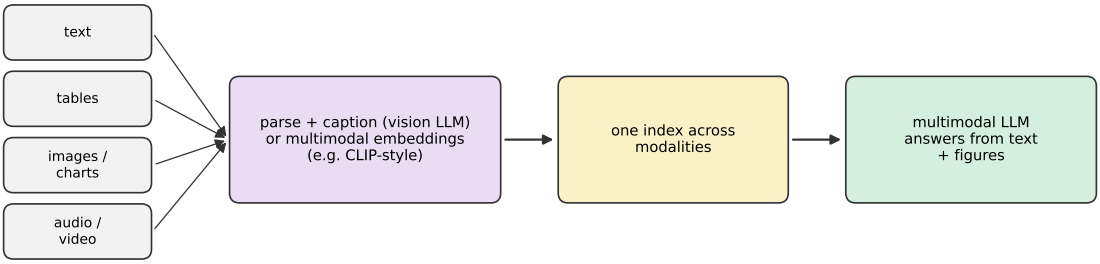


Figure 4.12: Retrieve across text, tables, images and more.

Options: convert images and tables to text descriptions with a vision model and index the text, or use multimodal embeddings that place images and text in the same vector space.

4.12 Other patterns worth knowing

- **RAPTOR** – builds a tree of recursive summaries (chunks → cluster summaries → higher summaries) and retrieves at whichever level fits.

- **Contextual retrieval** – prepend a short LLM-written description of where each chunk sits in its document before embedding it.
- **Multi-hop / iterative RAG** – retrieve, read, generate the next query, retrieve again.
- **Text-to-SQL "RAG"** – for structured data, generate a SQL query rather than retrieving text.
- **Cache-augmented generation (CAG)** – for small, stable knowledge bases, put everything into a long context (with prompt caching) instead of retrieving.

4.13 Comparison of RAG types

Type	Core idea	Best for	Trade-off
Naive	Embed, top-k, answer	Prototypes, simple FAQs	Brittle on hard queries
Advanced	Pre/post-retrieval optimisation	Most production apps	More components to tune
Modular	Swappable modules in a graph	Complex, evolving systems	Engineering effort
Hybrid	Keyword + vector (+ graph), fused	IDs, codes, jargon	Extra index, fusion tuning
Graph RAG	Knowledge graph of entities/relations	Multi-hop, relationships, global summaries	Expensive indexing
Agentic	Agent chooses tools and when to search	Multi-source, multi-step	Latency, cost, less predictable
Corrective (CRAG)	Grade docs, fall back if poor	Reliability, noisy corpora	Extra grader calls
Self-RAG	Model trained with reflection tokens	Research, self-critique	Needs special model
Adaptive	Route by question complexity	Mixed traffic, cost control	Router mistakes
Conversational	Rewrite follow-ups with history	Chatbots	Rewrite errors
Multimodal	Text + images + tables	Manuals, reports, slides	Parsing complexity

CHAPTER 5

LangChain

5.1 The LangChain ecosystem

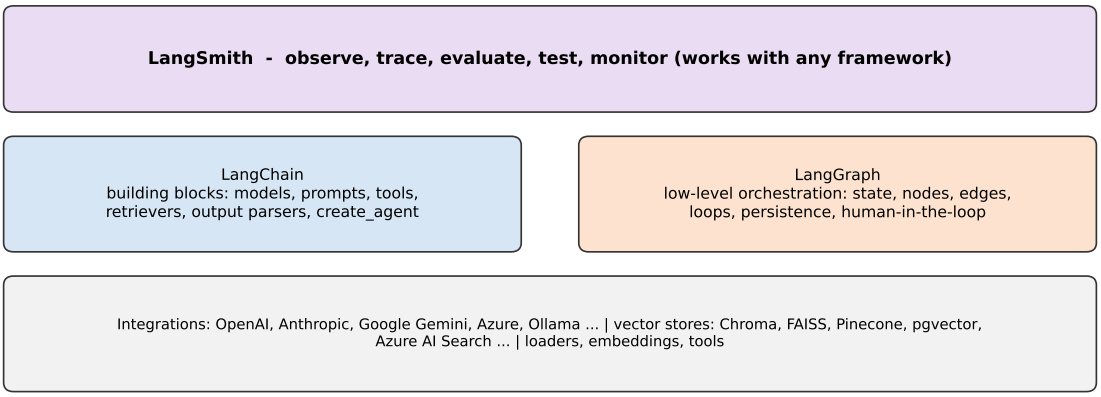


Figure 5.1: Three tools, three jobs: LangChain (building blocks), LangGraph (orchestration), LangSmith (observability and evaluation).

The idea in plain words

LangChain is a framework that gives you ready-made, interchangeable building blocks for LLM apps – so switching from OpenAI to Gemini, or from Chroma to Pinecone, is a one-line change instead of a rewrite.

Remember

When to use what: LangChain for straightforward pipelines (prompt → LLM → parse, a simple RAG chain) and quick agents; **LangGraph** when you need branching, loops, retries, multiple agents, persistence or human approval. LangChain's own agents run on LangGraph underneath.

5.2 Core components

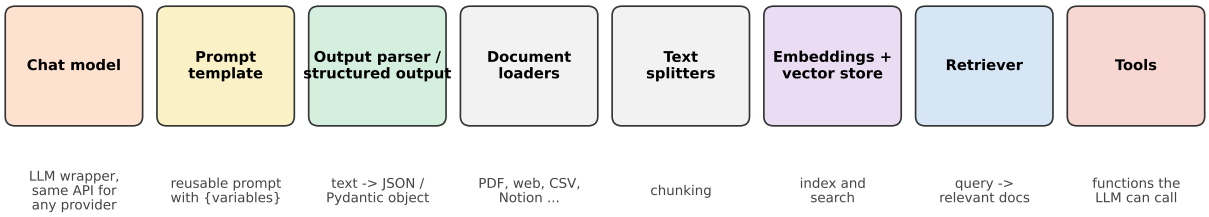


Figure 5.2: The main building blocks.

Chat models One interface (`invoke`, `stream`, `batch`) for any provider; messages have roles: `SystemMessage`, `HumanMessage`, `AIMessage`, `ToolMessage`.

Prompt templates Reusable prompts with variables: "Answer using {context}. Question: {question}".

Structured output Ask the model to return data matching a Pydantic schema (`with_structured_output`) instead of free text – essential for reliable apps.

Document loaders, text splitters Load sources and chunk them (Chapter 2).

Embeddings + vector stores Create vectors and store/search them.

Retrievers Anything that maps a query to documents: vector-store retriever (similarity or MMR), BM25, ensemble (hybrid), multi-query, parent-document, contextual-compression.

Tools Python functions the model can call; the docstring and type hints become the tool's description and schema – write them clearly.

5.3 LCEL and Runnables

The idea in plain words

Every component is a **Runnable** with the same methods. The **pipe operator** `|` connects them so the output of one becomes the input of the next – the LangChain Expression Language (LCEL).

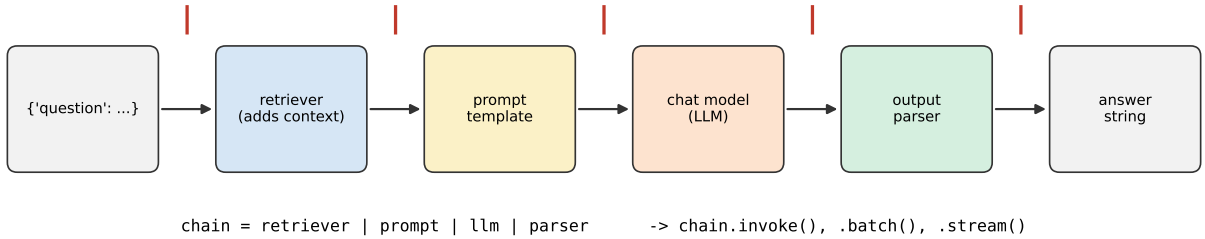


Figure 5.3: A RAG chain built with pipes.

Python (sketch)

```
prompt = ChatPromptTemplate.from_template(
    "Answer only from the context.\nContext: {context}\nQuestion: {question}")
chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
chain.invoke("How many leave days carry over?")
```

Every LCEL chain automatically supports streaming, batching, async and LangSmith tracing.

5.4 Agents in LangChain

In LangChain 1.x, `create_agent(model, tools, system_prompt=...)` builds a tool-calling agent (it runs as a LangGraph graph). **Middleware** plugs extra behaviour into the agent loop without rewriting it, for example:

- model and tool **retry** with exponential back-off, **model fallback** to a second provider;
- **summarisation** of long conversations near the token limit;
- **call limits** on model and tool calls (cost control, stops infinite loops);
- **PII detection** and redaction;
- **human-in-the-loop** approval before chosen tools run (Chapter 10).

Python (sketch)

```
agent = create_agent(
    model="openai:gpt-...",              # any supported provider
    tools=[search_policies, get_order_status],
    system_prompt="You are a helpful support agent.",
)
agent.invoke({"messages": [{"role": "user", "content": "Where is order 1042?"}]})
```

5.5 Memory

LLMs are stateless; "memory" means storing and re-sending context.

- **Short-term (conversation) memory** – the message history of one thread, trimmed or summarised to fit the context window. In LangGraph it lives in the state and is saved by a **checkpointer**.
- **Long-term memory** – facts that persist across conversations (user preferences, past decisions) in a **store** or database, retrieved when relevant.

CHAPTER 6

AI Agents

6.1 What is an AI agent?

The idea in plain words

An **agent** is an LLM that works towards a goal in a **loop**: it decides what to do next, uses **tools** (search, databases, APIs, code), looks at the results, and repeats until the job is done. A chatbot answers; an agent *acts*.

Real-life analogy

A chatbot is a knowledgeable receptionist who answers questions. An agent is an assistant who can also open the booking system, check availability, make the reservation and email you the confirmation – deciding the steps on their own.

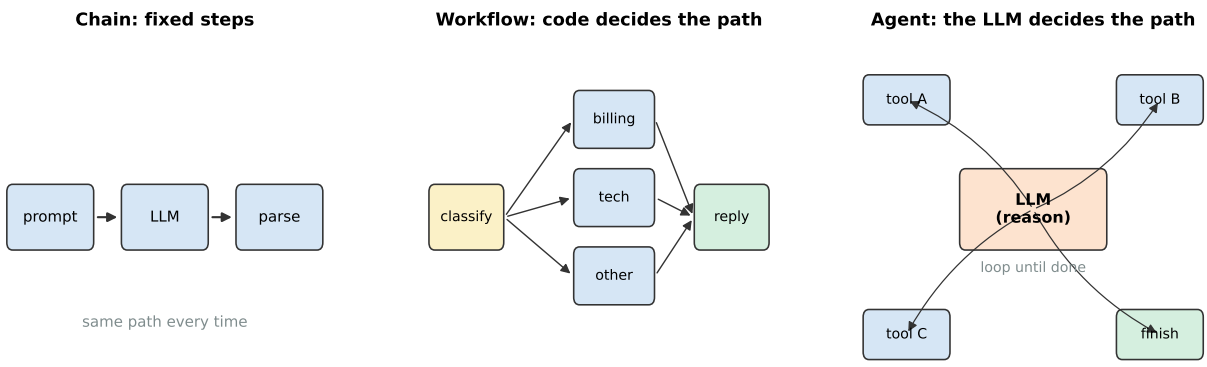


Figure 6.1: Chain, workflow and agent: who decides the path?

Remember

Use the simplest thing that works. A fixed chain or a code-controlled workflow is cheaper, faster and more predictable. Use an agent only when the steps genuinely can't be known in advance.

6.2 Anatomy of an agent

- **Model** – reasons and chooses actions.
- **Instructions** – role, goals, rules, output format, when to stop.
- **Tools** – each with a clear name, description and typed arguments. The model only *requests* a tool call; your code executes it.
- **Memory** – conversation state and long-term facts.
- **Guardrails** – step limits, permissions, validation, human approval.

6.3 Tool calling

MCP (Model Context Protocol) is an open standard for exposing tools and data sources to agents, so one tool server can be reused by many agent apps.

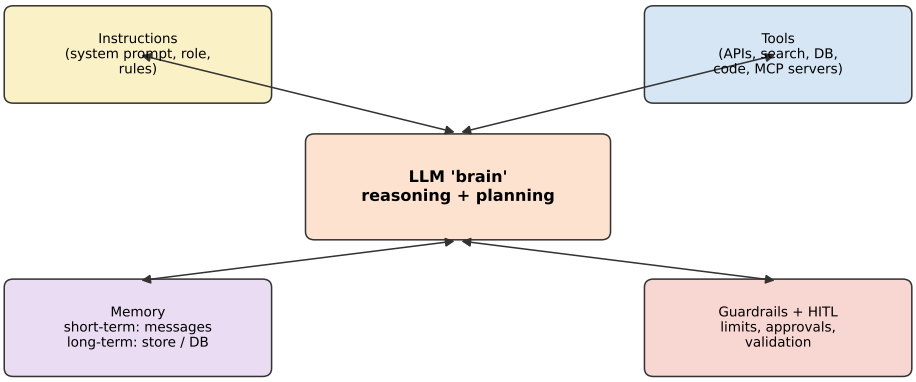


Figure 6.2: The parts of an agent.

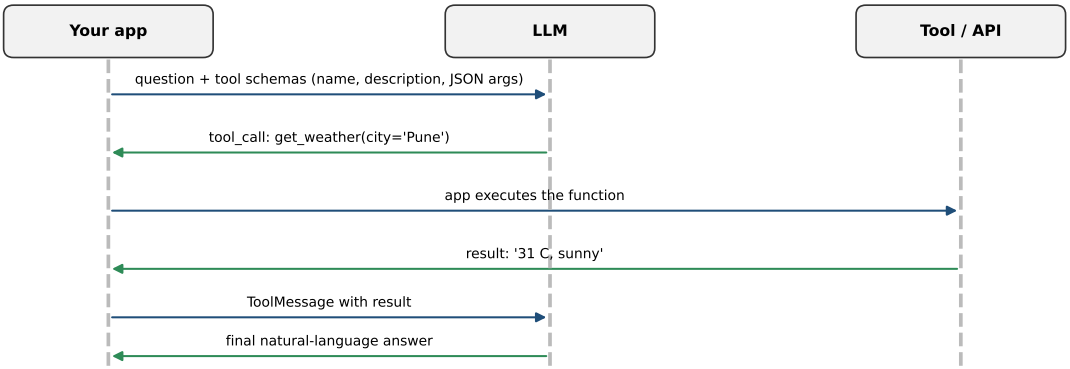


Figure 6.3: Tool (function) calling: the model returns a structured request; the application runs the tool and sends the result back.

6.4 The ReAct pattern

The idea in plain words

ReAct = Reason + Act. The agent alternates between *thinking* about what to do, *acting* (calling a tool) and *observing* the result, until it can answer.

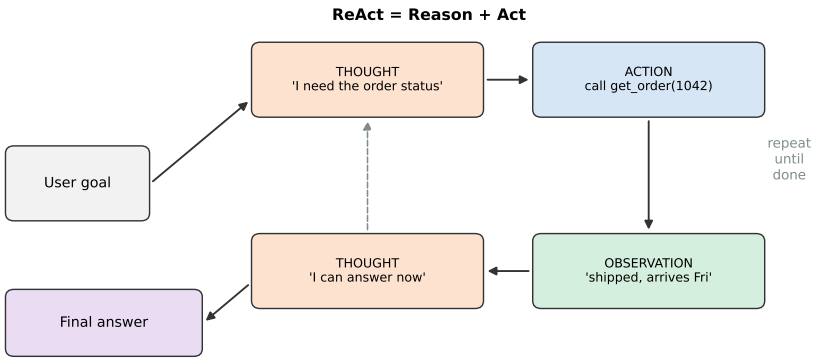


Figure 6.4: The ReAct loop.

Key equations

Agent loop (pseudo-code):

```
while steps < max_steps:
msg = llm(messages + tools)
if no tool_calls in msg:  return msg
for call in msg.tool_calls:  messages.append(run(call))
```

Always set **max_steps** (recursion limit) so a confused agent can't loop forever.

6.5 Common agent and workflow patterns

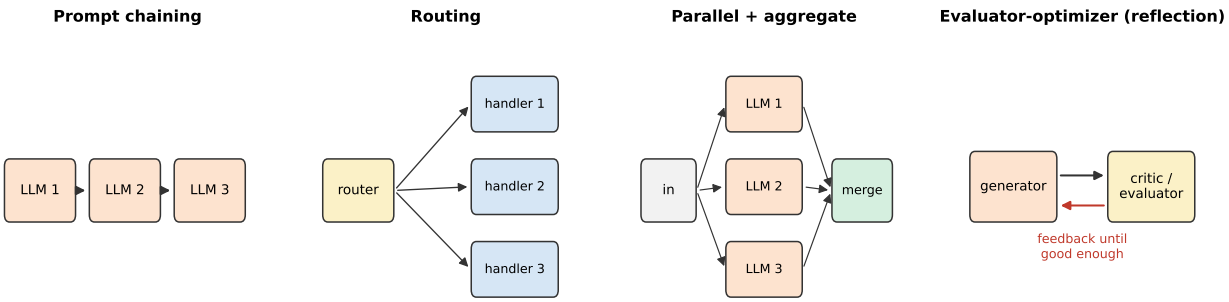


Figure 6.5: Building blocks of agentic systems.

Prompt chaining Fixed sequence of LLM calls (outline → draft → polish).

Routing Classify the input, send it to a specialised handler or model.

Parallelisation Run independent calls at once (e.g. three extractors), then aggregate.

Plan-and-execute One LLM call writes a plan; steps are executed (by code or a worker agent); re-plan if needed.

Reflection / evaluator–optimizer A critic reviews the output and the generator revises until it passes.

6.6 Multi-agent systems

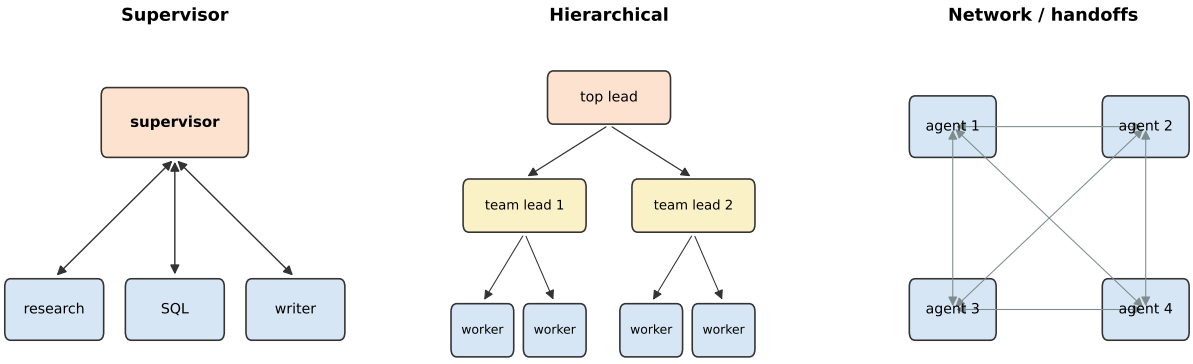


Figure 6.6: Three ways to organise several agents.

- **Supervisor** – a coordinator routes work to specialist agents and combines results (most common).
- **Hierarchical** – supervisors of supervisors for large tasks.
- **Network / handoffs** – agents pass control directly to each other.

Specialised agents get focused prompts and fewer tools, which usually improves accuracy – but each extra agent adds cost, latency and new failure points.

6.7 Reliability: retries, errors and limits

- **Retry with exponential back-off** for transient failures (rate limits, timeouts): wait 1 s, 2 s, 4 s . . . then give up gracefully.
- **Validate** tool arguments and LLM outputs with schemas; on failure, send the error back to the model so it can correct itself (limited attempts).
- **Fallbacks** – secondary model or a safe default answer.
- **Limits** – maximum steps, tool calls, tokens and time per request.
- **Keep decisions in code** where rules matter: let the LLM extract and draft, but let deterministic Python apply business rules (eligibility, prices, approvals).

Watch out

Agents fail quietly: they may call the wrong tool, loop, or reach the right answer by a wrong route. That's why agents need tracing (LangSmith) and trajectory evaluation (Chapter 9).

CHAPTER 7

LangGraph

7.1 What LangGraph is

The idea in plain words

LangGraph lets you draw your LLM application as a **graph**: **nodes** are steps (Python functions, LLM calls, tools), **edges** decide what runs next, and a shared **state** carries data between them. Unlike a simple chain, the graph can **branch, loop, retry, run steps in parallel, pause for a human, and remember** where it is.

Real-life analogy

A flowchart that actually runs – and that keeps a saved game after every move, so you can stop, resume, or rewind.

7.2 Core concepts

Concept	Meaning
State	A typed dictionary (`TypedDict` or Pydantic) shared by all nodes: messages, documents, flags, counters
Node	A function `state -> partial update`; returns only the keys it changes
Edge	A fixed transition A → B
Conditional edge	A function that reads the state and returns the name of the next node (branching, loops)
START / END	Special entry and exit points
Reducer	How an update merges into a key: overwrite (default) or e.g. `add_messages` to append
Compile	`builder.compile(checkpointer=...)` turns the builder into a runnable graph
Checkpointer	Saves state after each step per `thread_id` (memory, resume, HITL, time travel)
Command	Returned from a node to update state *and* jump to a node; also used to resume interrupts
Send	Fan-out: run a node many times in parallel (map-reduce)
Subgraph	A compiled graph used as a node inside another graph

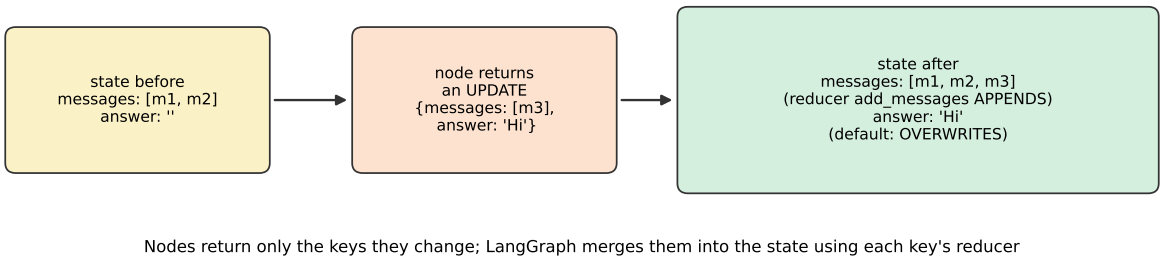


Figure 7.1: Nodes return partial updates; reducers merge them into the state.

7.3 The agent loop as a graph

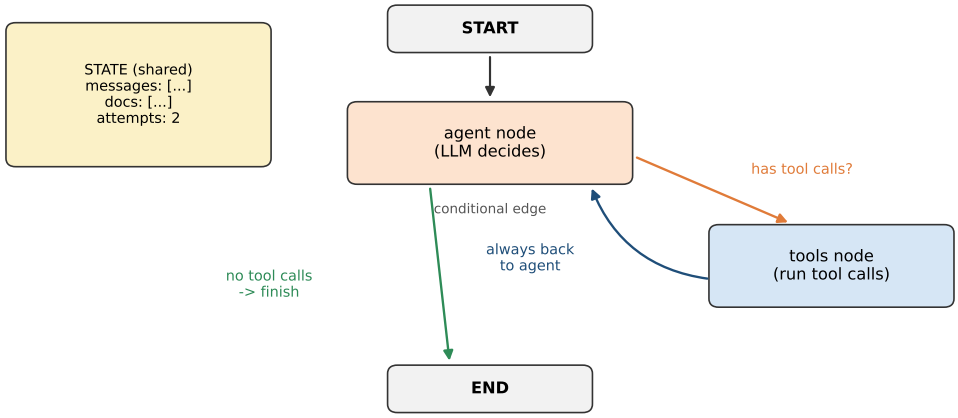


Figure 7.2: The classic tool-calling agent in LangGraph: the agent node either calls tools (and loops back) or finishes.

Python (sketch)

```
class State(TypedDict):
    messages: Annotated[list, add_messages]

def agent(state):  return {"messages": [llm_with_tools.invoke(state["messages"])]}
def route(state):  return "tools" if state["messages"][-1].tool_calls else END

builder = StateGraph(State)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route, ["tools", END])
builder.add_edge("tools", "agent")
graph = builder.compile(checkpointer=InMemorySaver())
graph.invoke({"messages": [("user", "hi")]},
             {"configurable": {"thread_id": "t1"}})
```

7.4 A Corrective-RAG graph

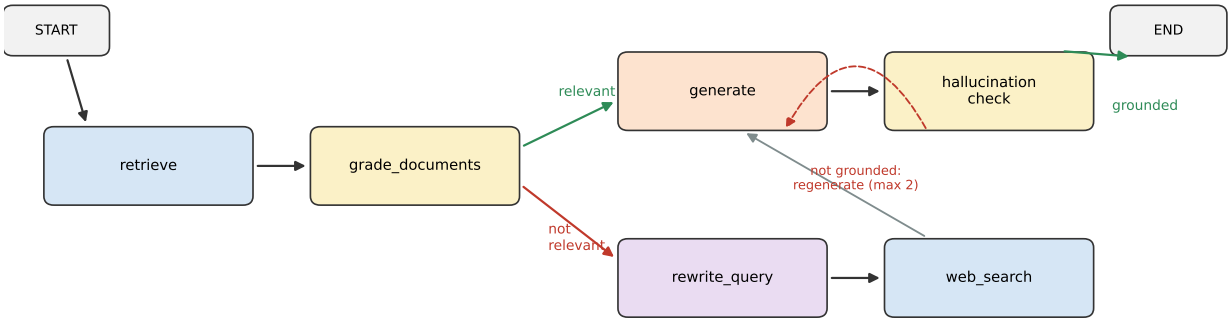


Figure 7.3: CRAG as a LangGraph: grading, query rewriting, web fallback and a bounded hallucination-check retry loop.

Keep a counter (e.g. `attempts`) in the state and stop after a maximum; LangGraph also enforces a `recursion_limit` on the number of steps.

7.5 Persistence and checkpoints

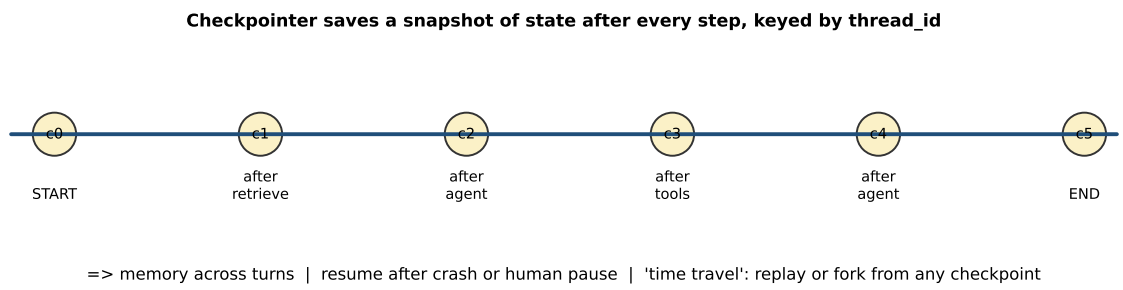


Figure 7.4: A checkpoint after each step, stored per thread.

- **Checkpointers**: `InMemorySaver` (development), `SqliteSaver`, `PostgresSaver` (production).
- **thread_id** identifies a conversation. Same thread = continue where you left off; new thread = fresh state.
- **Time travel**: inspect the state history, replay from an earlier checkpoint, or fork with an edited state – great for debugging.
- **Streaming**: stream node updates, full state values, or LLM tokens to the UI as the graph runs.

7.6 Human-in-the-loop in LangGraph

The idea in plain words

A node can **pause** the graph and ask a person for input. LangGraph saves the state with the checkpointer and waits – for seconds or days. When the person responds, you resume the same thread and the graph continues from exactly that point.

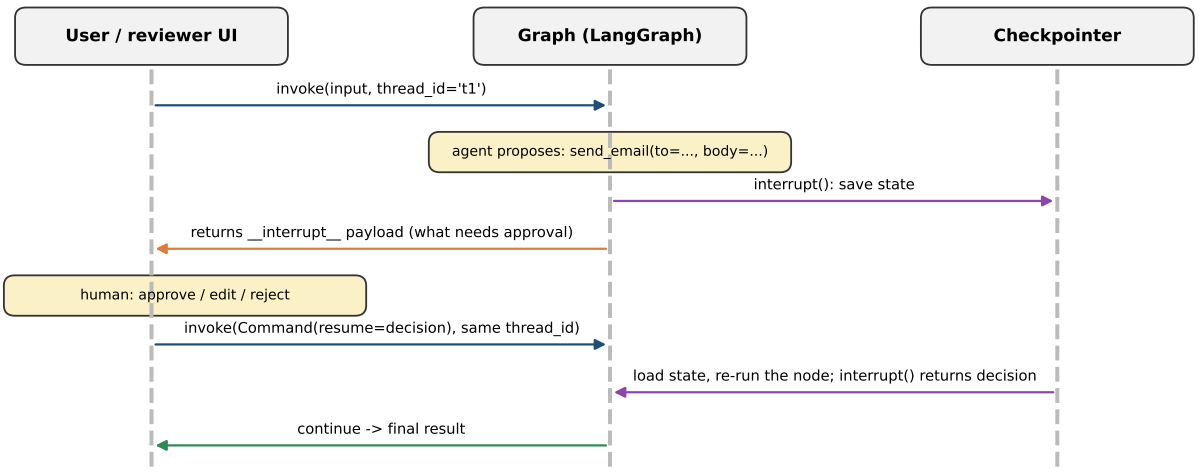


Figure 7.5: Interrupt and resume. The value passed to `Command(resume=...)` becomes the return value of `interrupt()` inside the node.

Python (sketch)

```
def approve_email(state):
    decision = interrupt({"action": "send_email", "draft": state["draft"]})
    if decision["type"] == "approve":
        return Command(goto="send")
    return Command(goto="revise", update={"feedback": decision.get("note")})

# caller side
graph.invoke(inputs, config)                          # pauses, returns __interrupt__
graph.invoke(Command(resume={"type": "approve"}), config)   # same thread_id
```

- **Dynamic interrupts** – call `interrupt(payload)` anywhere inside a node; can be conditional (only when risk is high).
- **Static breakpoints** – `compile(interrupt_before=["node"])` or `interrupt_after`; mainly for debugging, or for "always stop before this step".
- **Requirements** – a checkpointer, the same `thread_id` on resume, and a JSON-serialisable payload.

Watch out

On resume, the node that called `interrupt()` **runs again from its beginning**. Any side effect before the interrupt (sending an email, writing to a DB) will repeat – so put side effects after the interrupt or make them idempotent. Don't wrap `interrupt()` in a bare try/except, and don't reorder multiple interrupt calls inside one node.

CHAPTER 8

LangSmith: Observability & Evaluation Platform

8.1 Why observability?

The idea in plain words

An LLM app is a chain of non-deterministic steps. When an answer is wrong, you need to see *which* step went wrong: bad retrieval? wrong tool? prompt bug? slow model? **LangSmith** records every step of every run as a **trace** so you can inspect, debug, evaluate and monitor – for LangChain/LangGraph apps and for plain Python too.

Real-life analogy

A flight recorder (black box) for your AI app: after any incident you can replay exactly what happened, step by step.

8.2 Tracing

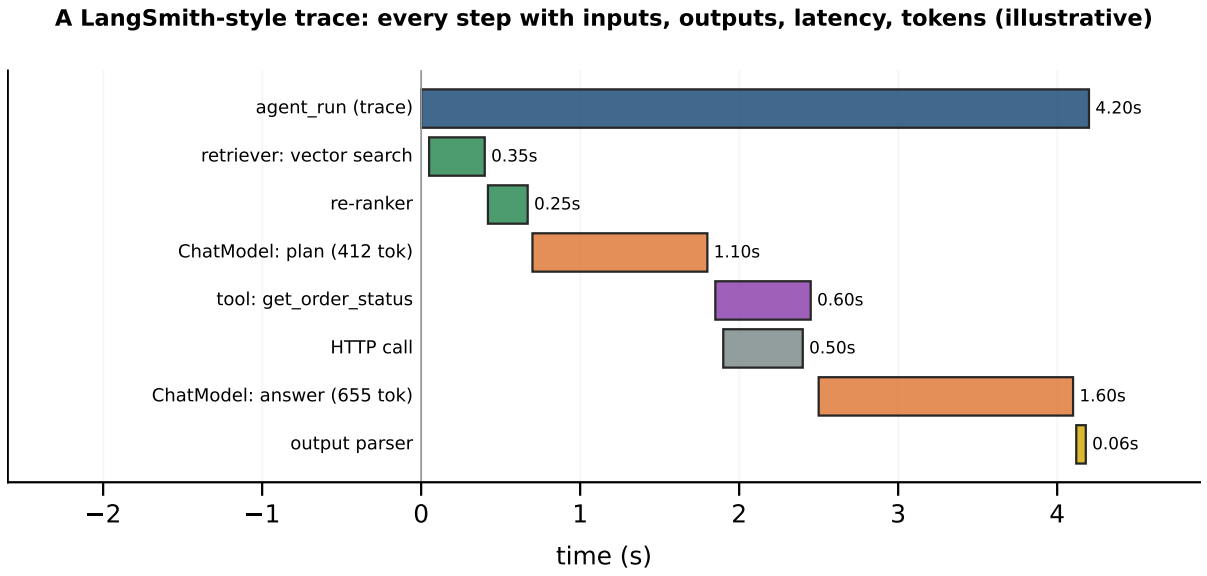


Figure 8.1: A trace is a tree of runs: each LLM call, retriever, tool and parser with inputs, outputs, latency, tokens and errors.

- **Run** – one step (an LLM call, a retriever call, a tool). **Trace** – all runs for one request. **Thread** – several traces in one conversation. **Project** – a collection of traces.
- **Turn it on**: set environment variables – `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY=...` (optionally `LANGSMITH_PROJECT`). LangChain and LangGraph are traced automatically; other code can use the `@traceable` decorator.
- Attach **metadata and tags** (user id, prompt version, model) to filter traces later.

8.3 What LangSmith offers

Feature	What it is for
Tracing & debugging	See every step, input, output, error, token count and latency
Datasets	Collections of examples: inputs + optional reference outputs + metadata
Evaluators	Functions that score outputs: code, LLM-as-judge, pairwise, human
Experiments	One run of your app over a dataset with evaluator scores; compare side by side
Prompt playground / hub	Edit, version and test prompts across models
Annotation queues	Send selected traces to human reviewers with a scoring rubric
Online evaluation	Automatically score a sample of live production traces
Monitoring & alerts	Dashboards for volume, errors, latency, cost and feedback over time
Studio	Visualise and step through LangGraph graphs, including interrupts

8.4 The evaluation workflow

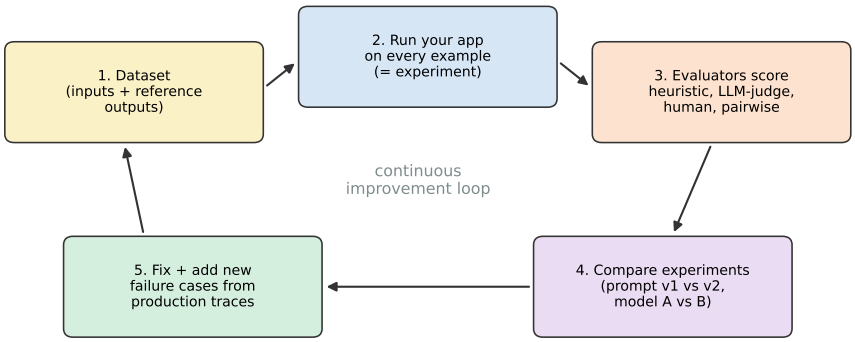


Figure 8.2: The improvement loop: dataset → experiment → evaluate → compare → fix and grow the dataset.

Python (sketch)

```
from langsmith import Client, evaluate

def correctness(outputs, reference_outputs):          # a custom evaluator
    return outputs["answer"].strip() == reference_outputs["answer"].strip()

def target(inputs):                                    # your app under test
    return {"answer": rag_chain.invoke(inputs["question"])}

evaluate(target, data="hr-policy-qa", evaluators=[correctness],
         experiment_prefix="hybrid-rerank-v2")
```

Remember

Start small: **10–50 carefully written examples** already catch most regressions. Grow the dataset with real failures from production traces and user feedback.

CHAPTER 9

Evaluating & Testing RAG and Agents

9.1 Why evaluation is different for LLM apps

Outputs are free text and can differ on every run, there is often more than one correct answer, and a correct-looking answer can hide a wrong process. So we combine **several evaluator types**, test **components and the whole system**, and run evaluation both **before release (offline)** and **in production (online)**.

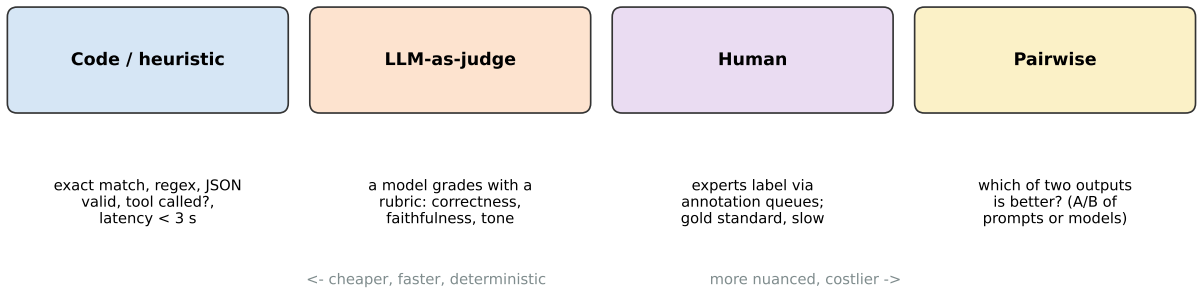


Figure 9.1: Four kinds of evaluators.

9.2 Evaluating RAG

9.2.1 The RAG triad

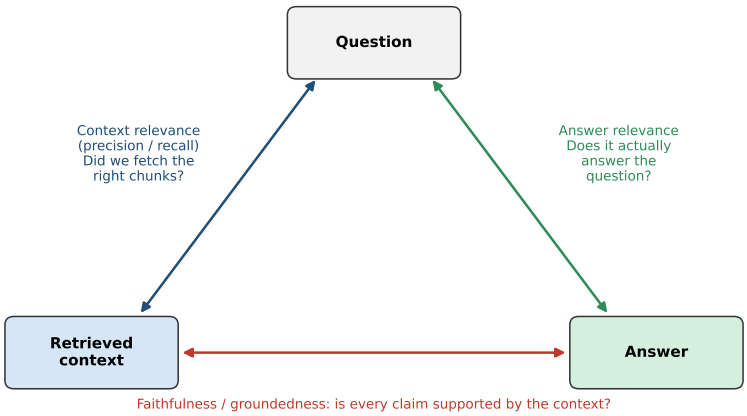


Figure 9.2: Three relationships to check in every RAG answer.

Metric	Question it answers	Needs reference?
Context precision	Are the retrieved chunks relevant (and ranked first)?	Optional
Context recall	Did we retrieve all the information needed?	Yes (reference answer)
Faithfulness / groundedness	Is every claim in the answer supported by the context?	No
Answer relevance	Does the answer address the question?	No
Answer correctness	Does it match the reference answer?	Yes

Frameworks such as **RAGAS**, DeepEval, TruLens and LangSmith evaluators implement these, usually with an LLM judge.

Key equations

$$\text{Faithfulness} = \frac{\#\text{ claims supported by the context}}{\#\text{ claims in the answer}}$$

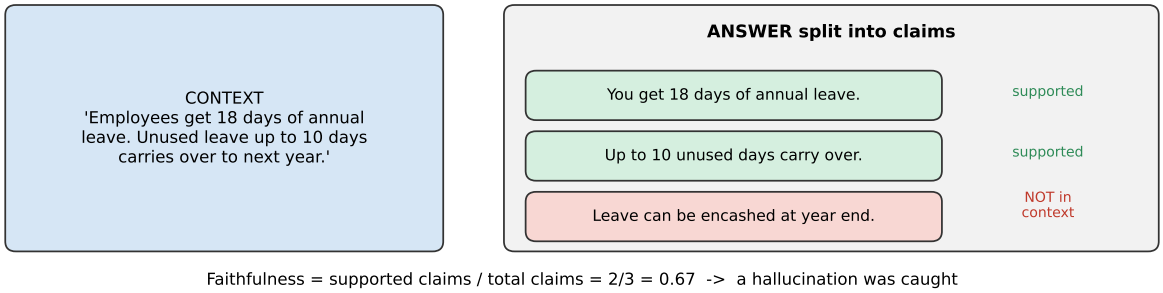


Figure 9.3: Faithfulness catches hallucinated claims even when the rest of the answer is right.

9.2.2 Retrieval metrics (need labelled relevant chunks)

Key equations

$$\text{Precision@}k = \frac{\#\text{relevant in top-}k}{k} \qquad \text{Recall@}k = \frac{\#\text{relevant in top-}k}{\#\text{relevant in total}} \qquad \text{Hit rate@}k = \text{share of queries with} \geq 1 \text{ relevant hit}$$

$$\text{MRR} = \frac{1}{|Q|}\sum_q \frac{1}{\text{rank of first relevant result}} \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad \text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}$$

Remember

Evaluate retrieval **separately** from generation. If context recall is low, no prompt will fix the answer – improve chunking, embeddings, hybrid search or re-ranking first.

9.3 LLM-as-judge

The idea in plain words

Use a (strong) LLM with a clear rubric to grade outputs – correctness against a reference, faithfulness to context, helpfulness, tone, safety.

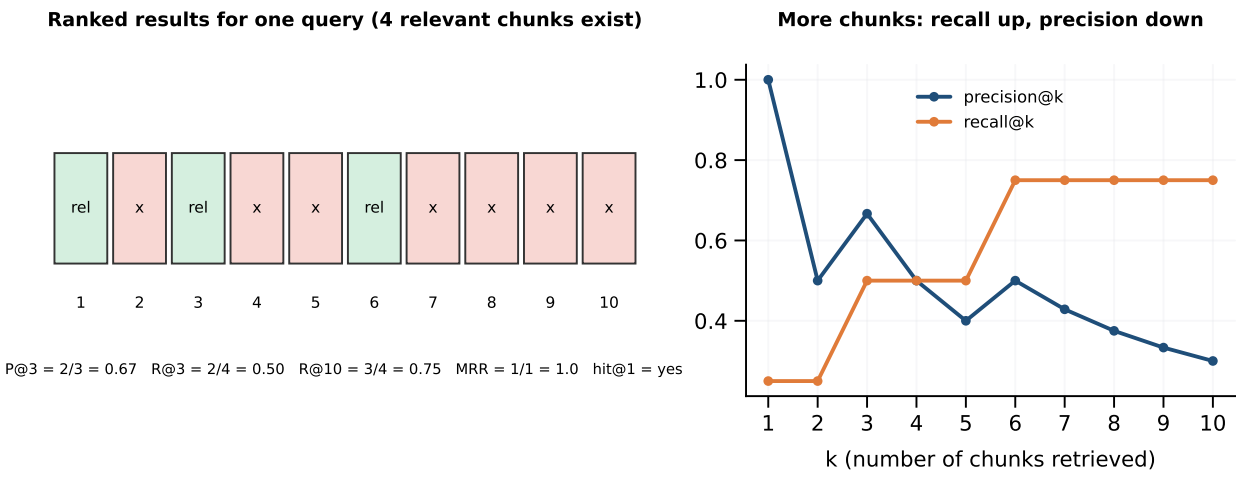


Figure 9.4: Worked example for one query, and how precision and recall move with k.

- Give a precise rubric and ask for a short reason *plus* a score (binary or 1–5); binary pass/fail is usually more consistent.
- Known biases: prefers longer answers, its own style, and the first option in pairwise comparisons – randomise order.
- **Calibrate** against human labels on a sample; check agreement before trusting it at scale.

9.4 Evaluating agents

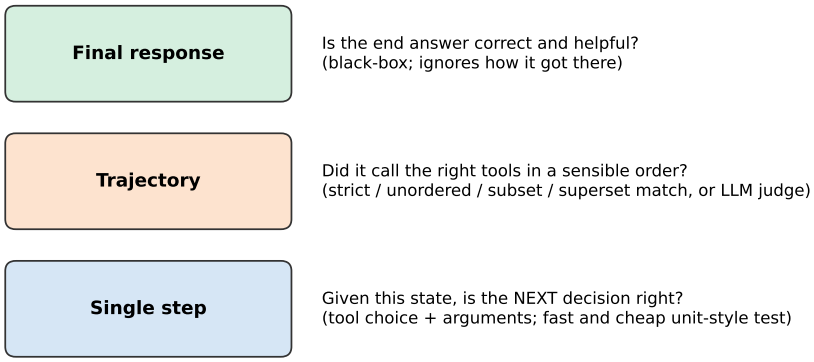


Figure 9.5: Agent evaluation at three levels.

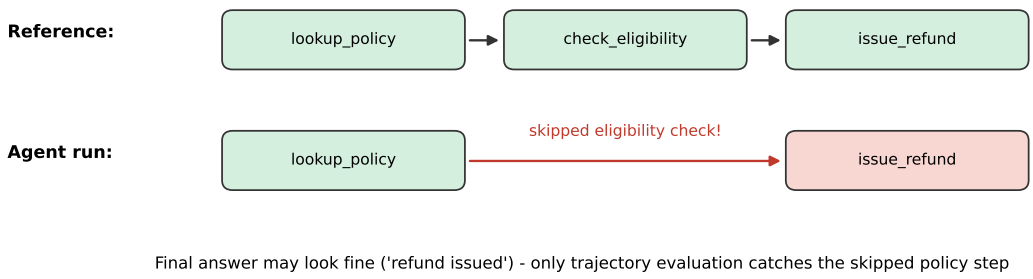


Figure 9.6: Why trajectories matter: a "correct" final answer can hide a skipped step.

- **Final response**: task success rate, correctness (reference or LLM-judge).

- **Trajectory**: compare the sequence of tool calls with a reference – *strict* (same order), *unordered* (same tools, any order), *subset* (no extra tools), *superset* (at least the required tools) – or ask an LLM judge whether the path was sensible and efficient. LangChain's `agentevals` package provides these evaluators.
- **Single step**: given a fixed state, did it choose the right tool with the right arguments?
- **Operational**: steps per task, tool-error rate, latency (p50/p95), tokens and cost per task, loop/timeout rate.
- **Multi-turn**: does the conversation stay coherent and reach the user's goal across turns?

Key equations

$$\text{Task success rate} = \frac{\#\text{tasks completed correctly}}{\#\text{tasks}} \qquad \text{Tool accuracy} = \frac{\#\text{correct tool calls}}{\#\text{tool calls}}$$

$$\text{Cost/task} = \frac{\text{total tokens} \times \text{price}}{\#\text{tasks}}$$

9.5 Testing strategy

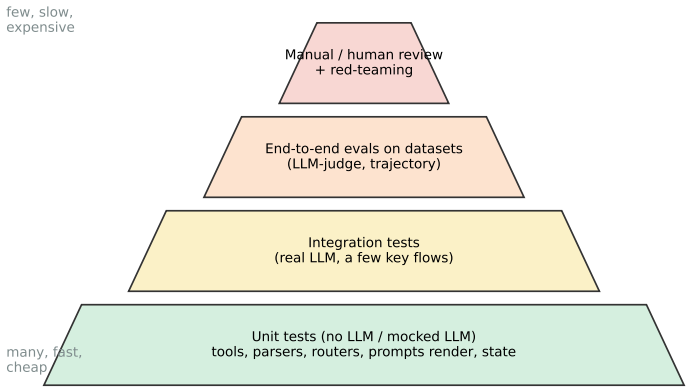


Figure 9.7: A testing pyramid for LLM applications.

Unit tests (no real LLM) Tools, parsers, routing functions, state reducers, prompt templates render correctly, business rules. Use a **fake/mocked LLM** or cached responses so tests are fast, free and deterministic.

Integration tests A few real calls through the full chain or graph: does a known question retrieve the expected document? Does the graph reach END?

Evaluation runs Dataset-based experiments with scores and thresholds; run in CI before deploying ("block if faithfulness drops below 0.85").

Regression tests Every bug found in production becomes a new dataset example.

Adversarial / red-team tests Prompt injection (direct and inside documents), jailbreaks, PII leakage, off-topic requests, out-of-scope questions that should get "I don't know".

Load and cost tests Latency under concurrency, rate limits, token budgets.

Watch out

Don't tune on your test set. Keep a held-out set, fix random seeds and temperature where possible, and run each evaluation more than once – LLM outputs vary between runs.

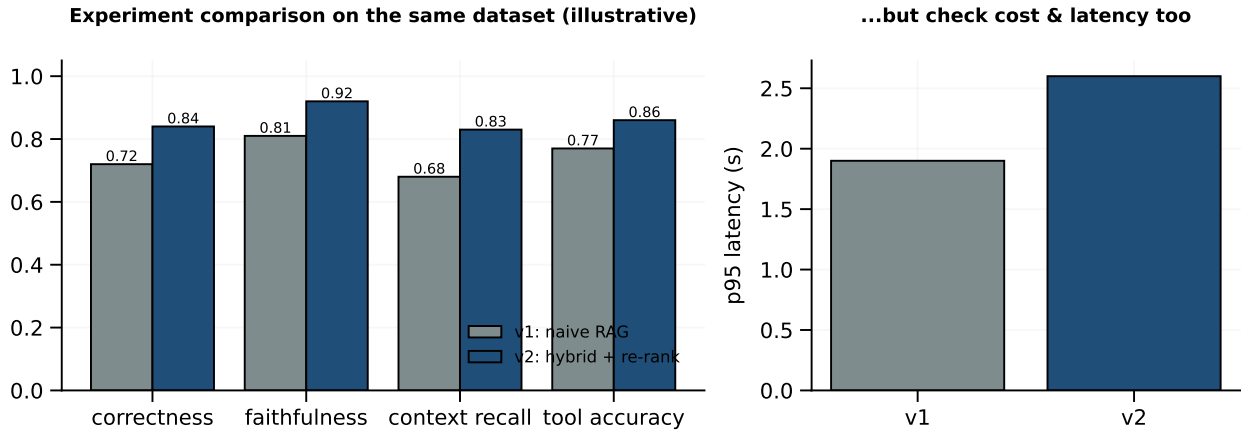


Figure 9.8: Compare versions on the same dataset – quality, and also latency and cost (illustrative numbers).

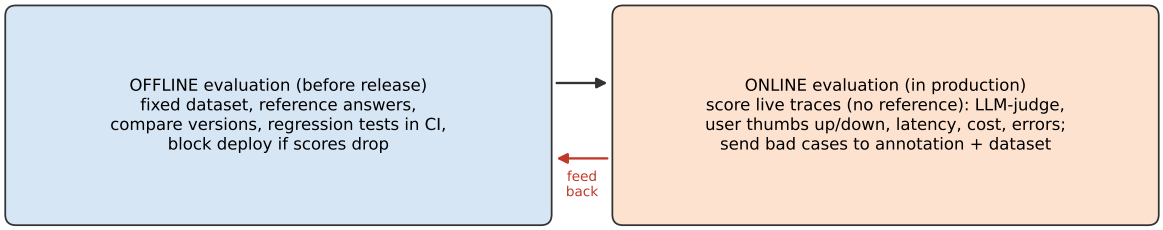


Figure 9.9: Offline and online evaluation feed each other.

CHAPTER 10

Human-in-the-Loop, Guardrails & Production

10.1 Human-in-the-loop (HITL)

The idea in plain words

Human-in-the-loop means a person can step into an AI workflow at chosen points – to **approve** a risky action, **correct** a draft, **supply** missing information, or **review** results that train better evaluations. The AI does the heavy lifting; people keep control where mistakes are costly.

Real-life analogy

A junior employee drafts the refund letter; a manager signs off before it is sent. The junior works fast, the manager prevents expensive errors.

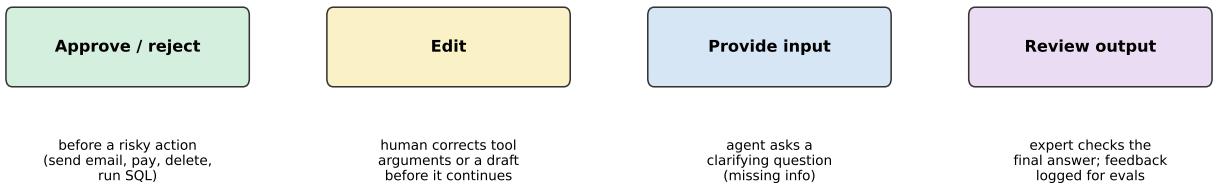


Figure 10.1: Four HITL patterns.

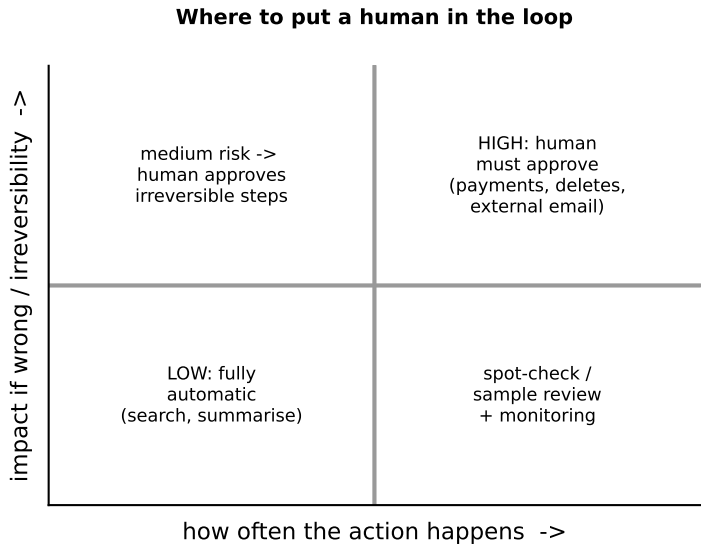


Figure 10.2: Decide where humans step in by impact and reversibility.

10.1.1 HITL at three levels

Level	What happens	Tooling
Runtime approval	Graph pauses before a sensitive step; human approves, edits, rejects or responds	LangGraph `interrupt()` + `Command(resume=...)`; LangChain `HumanInTheLoopMiddleware`
Escalation	Low confidence, high risk or user frustration → hand off to a human agent	Router / rule node, ticketing
Evaluation feedback	Experts label outputs; users give thumbs up/down	LangSmith annotation queues, feedback API

10.1.2 HITL middleware for agents (LangChain 1.x)

`HumanInTheLoopMiddleware(interrupt_on={...})` pauses the agent before chosen tools run. For each tool you set which decisions a reviewer may make:

Decision	Effect
approve	run the tool exactly as proposed
edit	change the tool's arguments, then run it (e.g. fix the email recipient)
reject	skip the tool and send feedback to the agent explaining why
respond	the human's message is returned as the tool result (for "ask the user" tools)

Python (sketch)

```
agent = create_agent(
    model=..., tools=[send_email, read_calendar, delete_file],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "send_email":   {"allowed_decisions": ["approve", "edit", "reject"]},
        "delete_file":  True,      # all decisions allowed
        "read_calendar": False,    # safe, auto-approved
    })],
    checkpointer=InMemorySaver(),  # required for pausing
)
```

Remember

HITL needs: a **checkpointer**, the **same thread_id** to resume, a clear **payload** so the reviewer knows what they're approving, and **idempotent** side effects. Tailor approvals by risk – too many approvals and people click "approve" without reading.

10.2 Guardrails and security

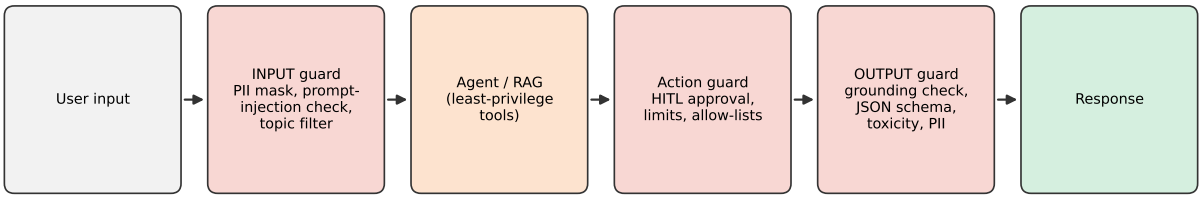


Figure 10.3: Layered guardrails around an agent or RAG app.

- **Prompt injection** – direct ("ignore your rules") or indirect (hidden in retrieved documents, web pages, emails). Defences: treat retrieved text as data, separate instructions from content, restrict tool permissions, require approval for side effects, test with attack prompts.
- **Least privilege** – read-only tools by default; scoped API keys; document-level access control in retrieval.
- **PII and secrets** – mask before sending to an external model; never log secrets; keep API keys in environment variables.
- **Output validation** – schema checks, grounding/citation checks, toxicity and policy filters.
- **Deterministic rules in code** – critical decisions (approve a refund, release a held case) should be plain Python checks, not something a prompt can talk its way past.

10.3 Production checklist

Area	Checklist
Data	Parsing verified, metadata and permissions on every chunk, re-index schedule, deletions propagated
Retrieval	Hybrid search, re-ranking, filters, evaluated recall@k
Generation	Grounding prompt, citations, "I don't know" path, structured outputs
Agents	Step/tool/token limits, retries with back-off, fallbacks, idempotent tools
HITL	Approval on high-risk actions, escalation path, reviewer UI
Evaluation	Golden dataset, CI evals with thresholds, online evals, regression cases from production
Observability	LangSmith tracing, dashboards for latency, cost, errors, feedback; alerts
Cost & speed	Caching (exact and semantic), smaller models for easy steps, streaming, batching
Security	Injection tests, least privilege, PII masking, secrets management, audit logs

CHAPTER 11

Cheat Sheet & Interview Revision

11.1 Formula sheet

Concept	Formula
Cosine similarity	$\dfrac{\mathbf{q}\cdot\mathbf{d}}{\|\mathbf{q}\|\|\mathbf{d}\|}$
Chunks with overlap	$\lceil (L-o)/(s-o) \rceil$
BM25 term score	$\text{IDF}\cdot\dfrac{f(k_1+1)}{f+k_1(1-b+b\|d\|/\text{avgdl})}$
RRF	$\sum_r 1/(60+\text{rank}_r)$
Weighted hybrid	$\alpha s_{\text{vec}} + (1-\alpha)s_{\text{BM25}}$
MMR	$\lambda\,\text{sim}(d,q) - (1-\lambda)\max_{s\in S}\text{sim}(d,s)$
Precision@k / Recall@k	relevant in top-k / k ; relevant in top-k / all relevant
MRR	mean of 1/rank of first relevant
nDCG@k	DCG/IDCG, $\text{DCG} = \sum \text{rel}_i/\log_2(i+1)$
Faithfulness	supported claims / total claims
Task success rate	tasks done correctly / tasks

11.2 Which RAG or agent design?

Situation	Choose
FAQ bot over a few clean documents	Naive RAG (maybe CAG if tiny)
Enterprise docs with codes, IDs, jargon	Hybrid search + re-ranker (advanced RAG)
Questions about relationships / whole corpus	Graph RAG
Noisy corpus, need reliability	Corrective RAG with a grader
Mix of easy and hard questions	Adaptive RAG router
Documents + database + web	Agentic RAG with tools
Chatbot with follow-ups	Conversational RAG (query rewriting + memory)
Fixed business process with LLM steps	LangGraph workflow, rules in code
Open-ended task, unknown steps	Tool-calling agent with limits
Risky actions (payments, emails, deletes)	Agent + HITL approval

11.3 Interview quick revision

What is RAG and why use it? Retrieve relevant passages from your data and add them to the prompt, so answers are grounded, current, private-data-aware and citable – without retraining the model.

RAG vs fine-tuning? RAG adds knowledge at run time and updates by re-indexing; fine-tuning changes behaviour/style in the weights. RAG for knowledge, fine-tuning for behaviour.

How do you choose chunk size? Balance precision (small) and context (large); start around a few hundred tokens with 10–20% overlap, then tune using retrieval evaluation.

What is hybrid search? Keyword (BM25) + vector search fused (e.g. RRF): exact terms plus meaning.

Bi-encoder vs cross-encoder? Bi-encoder embeds query and document separately (fast, indexable); cross-encoder reads both together (accurate, used to re-rank a short list).

What is MMR? Selects chunks that are relevant but not redundant.

What is Graph RAG? Builds a knowledge graph of entities and relationships and retrieves by traversing it or using community summaries – good for multi-hop and global questions.

CRAG vs Self-RAG? CRAG grades retrieved documents and falls back (e.g. web search) using any LLM; Self-RAG trains the model to emit reflection tokens for retrieval and self-critique.

What is agentic RAG? An agent decides when, where and how often to retrieve, using retrieval as one of several tools.

What is lost-in-the-middle? LLMs use information at the start and end of long contexts better than the middle; send fewer, re-ranked chunks with the best first.

LangChain vs LangGraph? LangChain = components and simple chains/agents; LangGraph = stateful graphs with branching, loops, persistence, multi-agent and HITL.

What is LCEL? Composing Runnables with the pipe operator: `prompt | llm | parser`.

What is state and a reducer in LangGraph? State is the shared typed data; a reducer defines how node updates merge (overwrite vs append, e.g. `add_messages`).

Why does LangGraph need a checkpointer? To save state per thread for memory, resuming, human-in-the-loop and time travel.

How does interrupt work? `interrupt(payload)` saves state and pauses; resume with `Command(resume=value)` on the same thread; the node re-runs from its start and `interrupt()` returns the value.

What is ReAct? Reason → act (tool) → observe, repeated until done.

How do you make agents reliable? Clear tools, schemas, retries with back-off, step limits, fallbacks, deterministic rules in code, HITL for risky actions, tracing and evaluation.

What does LangSmith do? Tracing, datasets, evaluators, experiments, prompt management, annotation queues, online evaluation and monitoring.

How do you evaluate RAG? Retrieval (precision/recall@k, MRR, nDCG, context precision/recall) and generation (faithfulness, answer relevance, correctness), often with RAGAS or LLM-as-judge.

How do you evaluate an agent? Final response, trajectory (tool sequence match or LLM judge), single-step decisions, plus success rate, cost, latency and step counts.

What are LLM-as-judge pitfalls? Verbosity, position and self-preference bias; mitigate with clear rubrics, binary scores, randomised order and calibration against humans.

How do you test without calling the LLM? Unit-test tools, parsers and routing with a fake or cached LLM; reserve real calls for integration tests and evaluation runs.

What is prompt injection and how do you defend? Malicious instructions in user input or retrieved content; treat content as data, least-privilege tools, HITL for side effects, output validation, red-team tests.

Further Reading

1. P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
2. Y. Gao et al. Retrieval-Augmented Generation for Large Language Models: A Survey. 2023 (naive / advanced / modular RAG).
3. D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft, 2024.
4. S. Yan et al. Corrective Retrieval Augmented Generation (CRAG). 2024. A. Asai et al. Self-RAG. 2023.
5. S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. 2022.
6. N. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. 2023.
7. S. Es et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation. 2023.
8. Anthropic, “Building effective agents” (workflow and agent patterns), 2024.
9. LangChain, LangGraph and LangSmith documentation (docs.langchain.com): interrupts, human-in-the-loop middleware, persistence, evaluation and trajectory evals.